Adaptive Threat Detection for Securing Cloud CI/CD Pipelines Using Evolving Neural Architectures

by

Sahil Ishan Tonmoy
22301612
S.M. Mashrafi Zaman
22301614
Shafayet Karim
22101507
Shifat Ahmed
22301582

A thesis submitted to the Department of Computer Science and Engineering
in partial fulfillment of the requirements for the degree of
B.Sc. in Computer Science and Engineering

Department of Computer Science and Engineering
Brac University
October 2025.

© 2024. Brac University
All rights reserved.

Declaration

It is hereby declared that

1. The thesis submitted is my/our own original work while completing degree at Brac University.
2. The thesis does not contain material previously published or written by a third party, except where this is appropriately cited through full and accurate referencing.
3. The thesis does not contain material which has been accepted, or submitted, for any other degree or diploma at a university or other institution.
4. We have acknowledged all main sources of help.

Student's Full Name & Signature:

Sahil Ishan Tonmoy	S.M. Mashrafi Zaman
22301612	22301614
Shafayet Karim	Shifat Ahmed
22101507	22301582

Approval

The thesis/project titled "Adaptive Threat Detection for Securing Cloud CI/CD Pipelines Using Evolving Neural Architectures" submitted by

1. Sahil Ishan Tonmoy (22301612)
2. S.M. Mashrafi Zaman (22301614)
3. Shafayet Karim (22101507)
4. Shifat Ahmed (22301582)

of [Summer], [2025] has been accepted as satisfactory in partial fulfillment of the requirement for the degree of B.Sc. in Computer Science in [Summer] Year.

Examining Committee:

Supervisor:
(Member)

Mr. Partha Bhoumik

Lecturer
Department of Computer Science and Engineering
Brac University

Co-Supervisor:
(Member)

Name of Co-Supervisor

Designation
Department
Institution

Thesis Coordinator:
(Member)

Md. Golam Rabiul Alam

Professor
Department of Computer Science and Engineering
Brac University

Head of Department:
(Chair)

Sadia Hamid Kazi

Chairperson and Associate Professor
Department of Computer Science and Engineering
Brac University

Ethics Statement (Optional)

This is optional, if you don't have an ethics statement then omit this page

Abstract

Cloud computing pipelines, particularly Continuous Integration and Continuous Deployment (CI/CD) environments, are essential for modern software development due to their flexibility, scalability, and automation. However, these dynamic cloud infrastructures face continuously evolving cyber threats, including polymorphic and metamorphic malware, which traditional static detection systems struggle to handle. Existing systems rely on fixed architectures, require new threat samples for retraining, and lack self-learning capabilities in sync with the pipeline, often resulting in high false positives, operational delays, and potential data breaches or financial loss. To address these limitations, this research proposes an adaptive malware detection system for cloud CI/CD pipelines. The system uses deep neural networks to perform both static and behavioral analysis of pipeline artifacts, specifically compiled executable binaries. A key feature is its self-evolving capability, where the neural network can continuously update its parameters dynamically in response to new behaviors, enabling the system to adapt to previously unseen threats. Unlike post-deployment scanners, this model is embedded directly in the pipeline, allowing real-time monitoring and detection. Additionally, the system includes a threat mutation module that generates modified versions of known malware and injects them into the training pipeline. This allows the model to learn from controlled adversarial samples while distinguishing them from external threats, drastically reducing false negatives and maintaining resilience against polymorphic evasion techniques. By continuously learning from both normal operational patterns and self-generated threats, the system maintains resilience against evolving malware and strengthens the security of cloud pipelines. The expected contribution of this research is a self-adaptive neural framework capable of evolving with changing threat behavior and providing proactive, context-aware protection for cloud CI/CD pipelines, addressing the shortcomings of current static detection systems, and enhancing the reliability, efficiency, and security of modern cloud software delivery.

Keywords: Adaptive Evolving Neural Security Architectures, Cloud-Native CI/CD Security, Polymorphic and Metamorphic Malware Detection, Multimodal Feature Fusion, CNN-BiLSTM Architecture, Adversarial Training, DevSecOps Automation

Dedication (Optional)

A dedication is the expression of friendly connection or thanks by the author towards another person. It can occupy one or multiple lines depending on its importance. You can remove this page if you want.

Acknowledgement

Table of Contents

Declaration i

Approval ii

Ethics Statement iv

Abstract v

Dedication vi

Acknowledgment vii

Table of Contents viii

List of Figures xi

List of Tables xii

Nomenclature xiv

1 Introduction 1
- 1.1 Background . . . 2
- 1.2 Rational of the Study or Motivation . . . 2
- 1.3 Problem Statement . . . 3
- 1.4 Objectives . . . 3
- 1.5 Methodology . . . 4
- 1.6 Scope and Limitations . . . 5
  - 1.6.1 Scope of the Research . . . 5
  - 1.6.2 Research Challenges . . . 5
- 1.7 Team Overview . . . 5
- 1.8 Key Learnings and Insights . . . 6

2 Literature Review 9
- 2.1 Preliminaries . . . 9
  - 2.1.1 Adaptive Evolving Neural Security Architectures . . . 9
  - 2.1.2 Cloud-Native CI/CD Security and Real-Time Defense . . . 9
  - 2.1.3 Polymorphic and Metamorphic Malware Detection . . . 9
  - 2.1.4 Behavioral and Code-Level Anomaly Detection . . . 9
  - 2.1.5 Static and Dynamic Threat Analysis (SAST/DAST) . . . 10
  - 2.1.6 Adversarial Training . . . 10

2.1.7 Red Team Simulation . . . 10
2.2 Review of Existing Research . . . 10
2.2.1 Adaptive AI and Neural Architecture Design . . . 10
2.2.2 Cloud and CI/CD Pipeline Security . . . 12
2.2.3 Threat Characterization and Detection . . . 12
2.2.4 Analytical and Testing Methodologies (SAST/DAST) . . . 15
2.2.5 Adversarial and Simulation-Based Defense . . . 15
2.3 Summary of Key Findings . . . 17

3 Requirements, Impacts and Constraintsn 20
3.1 Project Management Plan . . . 20
3.2 Risk Management . . . 20
3.3 System Specification . . . 21
3.3.1 Threat model . . . 21
3.4 Societal Impact . . . 22
3.5 Ethical Issues . . . 23

4 System Design and Implementation 24
4.1 Methodology Overview . . . 24
4.2 Data Collection . . . 25
4.2.1 Data Cleaning . . . 26
4.2.2 Data Transformation . . . 26
4.2.3 Data Integration . . . 26
4.2.4 Data Reduction . . . 27
4.2.5 Summary of Preprocessed Data . . . 27
4.3 System Architecture . . . 27
4.4 Implementation Details . . . 29
4.4.1 Developing Environment & Technology Stack . . . 29
4.4.2 System Architecture Implementation . . . 30
4.4.3 Operational Logic & Defending against Adversarial Attacks . 32

5 Result Analysis 34
5.1 Evaluation Metrics . . . 34
5.2 Performance Evaluation . . . 35
5.3 Analysis of Design Solutions . . . 36
5.3.1 Feature Fusion Strategies . . . 36
5.3.2 Statistical Analysis of Feature Fusion Strategies . . . 38
5.3.3 Adversarial Mutation Strategies . . . 41
5.3.4 Adversarial Resilience Analysis . . . 43
5.4 Final Design Adjustments . . . 47
5.4.1 Optimized Concatenation Architecture . . . 47
5.4.2 Dynamic Insert Mutation Engine . . . 47
5.4.3 Weighted Class Balancing . . . 47
5.4.4 Learning Rate Adjustments and Focal Loss Calibration . . . 48
5.5 Computational Latency and Architectural Asynchrony . . . 48
5.6 Comparisons and Relationships . . . 49
5.7 Discussions . . . 50

6 Conclusion **53**
6.1 Summary of Findings 53
6.2 Contributions to the Field 54
6.3 Recommendations for Future Work 55
6.3.1 npm Supply Chain Attack Detection 55
6.3.2 Graph Neural Networks for Dependency Analysis 55
6.3.3 Continual Learning 55
6.3.4 Multi-Ecosystem Support 55
6.3.5 Adversarial Attacks on the Static Branch (CNN) 56
6.3.6 Automated Remediation and Self-Healing Pipelines 56

Bibliography **59**

List of Figures

1.1 Research Overview 4

3.1 STRIDE Threat Model 21

4.1 Methodology Diagram 25
4.2 Component Communication Flow and Data Flow 28
4.3 Input modalities visualization. Left: Entropy-based visualization of a malware binary (CNN input). Right: Integer-encoded sequence of API calls (LSTM input). 31

5.1 Classification metrics across all four evaluated fusion methods. 37
5.2 ROC curves comparing the detection trade-offs across all four evaluated fusion schemes. 38
5.3 Precision-Recall curves across all four fusion strategies, highlighting the superior simultaneous precision and recall of the Concatenation method. 39
5.4 Radar chart comparing the multidimensional performance profile of each fusion architecture across five key metrics. 40
5.5 Confusion matrix heatmaps for each fusion architecture configuration (formatted in a 2×2 grid to prevent cramping). 41
5.6 Model detection rates under random insertion attacks across different mutation rates. 45
5.7 Model detection rates under random replacement attacks across different mutation rates. 45
5.8 Model detection rates under noise append attacks, showing high stability across all configurations. 46
5.9 Logarithmic bar chart highlighting the massive disparity between local inference latency (milliseconds) and retraining latency (minutes). . 49

List of Tables

1.1 Team Members and Primary Contributions 6

2.1 Strategies and features found in papers 17

4.1 Summary of sample counts, class balances, and branch tensor dimensions in the preprocessed multi-modal dataset. 27

5.1 Per-class classification report metrics for the baseline Dual-Branch model. 36
5.2 Confusion matrix classification counts for the baseline Dual-Branch model. 36
5.3 Performance comparison of feature fusion strategies. 37
5.4 Detection Rate (Recall) of training mutation strategies against evasion attacks. 42
5.5 Accuracy of training mutation strategies against evasion attacks. . . . 42
5.6 Precision of training mutation strategies against evasion attacks. . . . 42
5.7 Robustness evaluation matrix showing cross-protection accuracies (%) of different model configurations across evasion datasets (mutation rate $p = 0.2$). 43
5.8 Robustness decay of hardened model configurations under Random Insert evasion attacks at varying mutation rates. 44
5.9 Robustness decay of hardened model configurations under Random Replace evasion attacks at varying mutation rates. 44
5.10 Confusion matrix classification counts for the Insert-Hardened model, demonstrating 100% Recall with zero False Negatives. 46
5.11 Execution latency comparison across different processing modes. . . . 49
5.12 Benchmarking comparison of the proposed model configurations against existing literature. 50

Nomenclature

The next list describes several symbols & abbreviation that will be later used within the body of the document

AI Artificial Intelligence

API Application Programming Interface

AST Abstract Syntax Tree

AUC Area Under the ROC Curve

BiLSTM Bidirectional Long Short-Term Memory

CI/CD Continuous Integration and Continuous Deployment

CNN Convolutional Neural Network

CPU Central Processing Unit

DAST Dynamic Application Security Testing

DL Deep Learning

FN False Negative

FP False Positive

GAT Graph Attention Network

GCN Graph Convolutional Network

GNN Graph Neural Network

GPU Graphics Processing Unit

IDE Integrated Development Environment

ML Machine Learning

PE Portable Executable

PR Precision-Recall

ROC Receiver Operating Characteristic

SAST Static Application Security Testing

STRIDE Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege

TN True Negative

TP True Positive

XAI Explainable Artificial Intelligence

Chapter 1

Introduction

It's already been two decades since we stepped into the 21st century. If we look at computing infrastructure via virtualized platforms, cloud computing has been providing on-demand and elastic access to it. For the contemporary digital landscape, it has become an essential component. Opportunity to accelerate development, reduce costs, and improve productivity in cloud computing has been given to the enterprises. Cloud computing has become the backbone of modern digital operations. Building, testing, and releasing software is now being automated by continuous integration and continuous deployment. This automation has become the key part in the rapid development and deployment of software. Consistent performance, faster delivery, and collaboration are being ensured after the integration with cloud platforms. Compromised dependencies, malicious code injections, and tampered build images are now part of common threats. The dynamic and interconnected nature of cloud CI/CD pipelines makes them vulnerable to cyberattacks. Financial losses, data breaches, and service disruption can be caused by these attacks.

Current threat detection systems are often limited by relying on static analysis, which only identifies known threats by searching for predefined malicious signatures or patterns in code. As a static system needs frequent manual updates, it causes poor adaptability, delays, and false positives. Static analysis is mainly effective for known attacks, but it fails against metamorphic or polymorphic malware that constantly changes to avoid detection. The research proposes a dynamic, adaptive threat detection framework tailored for cloud CI/CD environments. It uses evolving neural architectures that can learn and adapt their internal weights in real time as new threats appear. Our system proposes to operate within the CI/CD pipeline (not after deployment) for continuous monitoring and instant threat response. The system combines static analysis, behavioral analysis, and a self-learning module to generate and examine mutated malware samples. It can more accurately distinguish real threats from benign code variations. So our proposed system will enhance the resilience and adaptability of CI/CD pipelines. It will represent a major advancement toward proactive, intelligent cybersecurity in cloud-based development environments.

1.1 Background

A single line of code emanates from a developer's laptop on the day it goes out and hurtles down cloud pipelines with unheard-of velocity. And as it goes along its way, it spins up services that didn't exist before and packages them on-the-fly into systems so complex they will be gone hours after. Yet while this nimbleness is characteristic of modern software delivery, that same swiftness also makes it difficult to distinguish the routine from the emergent risk: a version bump might hide a compromised dependency; an image may appear identical but with differences, or maybe a particular step works in one way today and another tomorrow. We must be automating all the things and killing alert noise with hammers so providers are deaf for weekly peaks while we miss once-a-month outbound requests. When security checks are initiated, they would halt the forward progress, either causing a delay in releases or getting permissive-based trust erosion to happen system-wide.

This research tackles this tension: pipelines change continuously, whereas defenses update infrequently. This is not another static control point. What we need is an adaptable framework to treat the pipeline as a dynamic organism, one capable of learning how "normal" moves across time and discerning live drift from stale change. The aim is to protect the delivery speed while at the same time maintaining resiliency, forming a defense that merges with day-to-day workflows, observing behavioral patterns across codes, and raising its level of understanding in every build iteration.

1.2 Rational of the Study or Motivation

Cloud CI/CD systems update software very quickly, but traditional security tools can't keep up. Old methods rely on fixed signatures or run scans only after deployment, so they often miss new or changing (polymorphic) malware, raise too many false alarms, and slow down releases.

This research builds a self-learning neural detector directly into the CI/CD pipeline. It combines both static and behavior-based analysis and keeps improving itself through a safe threat-mutation training loop. This makes detection faster, smarter, and more adaptable in real time.

The main goal is to make security learn and adapt as fast as the pipeline changes - and to prove that this approach performs better than existing threat detection models from literature in accuracy, adaptability, and efficiency.

Industry view: As software delivery becomes faster and more automated, security must become part of the process, not a separate checkpoint. This work builds a smart, built-in security gate that protects the system without slowing it down.

Practical view: Old security tools are slow and rigid. My approach uses a self-updating neural model and safe threat simulations to catch new types of attacks, reduce false positives, and keep the release process smooth.

1.3 Problem Statement

Static detection is inefficient in cloud CI/CD pipelines because they are scalable and dynamic. Current fixed systems lack embedded self-adaptive learning, which results in delays, errors, and vulnerability to new attacks. This is necessary for real-time analysis of constantly changing polymorphic threats.

1.4 Objectives

Below are the specific objectives that guide this research. They focus on designing an adaptive defense that learns in real time while fitting seamlessly into modern cloud CI/CD workflows.

1. **Design an architecture for a self-adaptive neural network**: We want to create a neural network model that dynamically modifies its internal weights and parameters to real-time changes in malware behavior.
2. **Integrate Threat Mutation for Adversarial Learning**: We want to implement an automated malware mutation module that creates a customized adversarial system so that it can effectively identify evolving threats.
3. **Enhance Pipeline Security through Continuous Self-Evolution**: There can be both mutated threats and regular operational behavior, so we will allow for continuous learning from both these threats to provide strong security against the newly identified polymorphic and metamorphic malware.
4. **Establish CI/CD Pipelines with Real-Time Monitoring**: At the industry level, embed the adaptive detection system directly into the CI/CD pipeline so that it can perform in real time without any delay in deployment.
5. **Evaluate and Benchmark the Adaptive Detection Framework**: Assess system performance using real-world datasets and benchmark it against existing threat detection models from literature to validate improvements in adaptability, precision, and efficiency.

1.5 Methodology

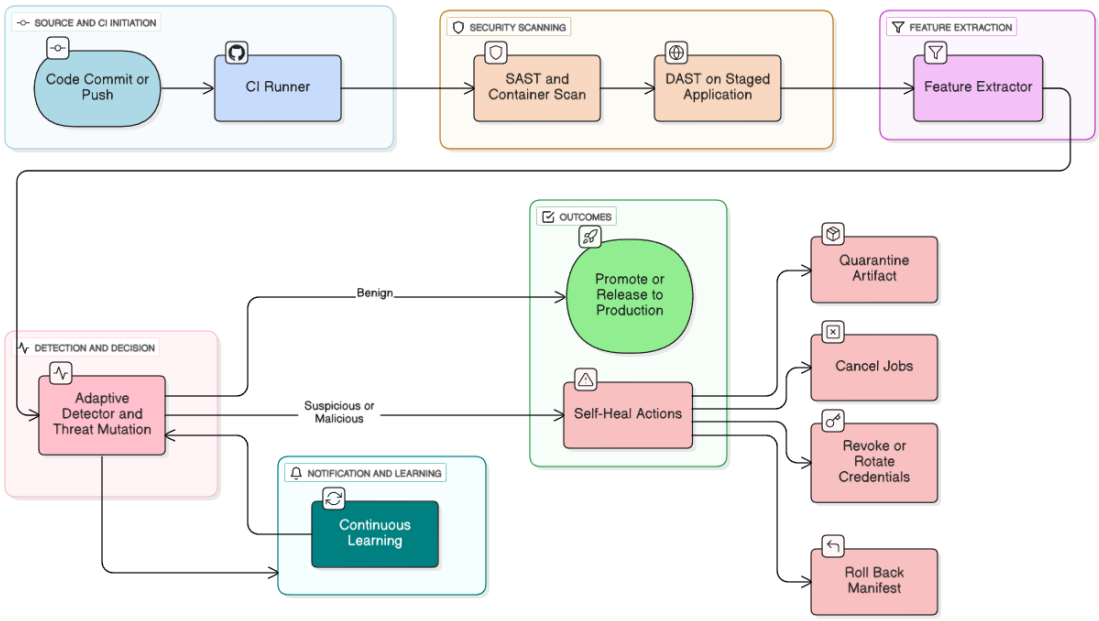


Figure 1.1: Research Overview

Our proposed system kicks into action automatically whenever a developer makes a code commit. Utilizing a dedicated Git `pre-commit` hook, it intercepts staged executable binaries before they can compromise the upstream repository. The system then runs a dual-modal security check: a static analysis that converts the binary into a structural image, and a dynamic analysis that extracts its sequential API execution behaviors. These modalities are fed into a dual-branch neural network (combining a CNN and a BiLSTM) designed to accurately spot complex threats.
What makes this system special is its Continuous Adaptation Engine. It keeps learning and getting better over time by utilizing adversarial hardening. During training, the system intercepts malware and creates slightly mutated, adversarial versions of the threat (such as inserting random API calls) to challenge itself. It uses these mutations to continuously retrain its weights via an asynchronous cloud pipeline, improving its detection boundaries against polymorphic malware.
Every code change is carefully reviewed by this local detector. If it's safe, the commit is allowed and the code moves forward smoothly into production. But if the neural network predicts a threat, the system jumps into immediate automated remediation: it forcefully aborts the commit process, blocks the CI/CD pipeline, and isolates the problematic files into a secure quarantine.
All of these steps—the multi-modal scan results, detection decisions, protective quarantines, and adversarial training loops—feed into a continuous learning process. This means the system is always evolving, getting smarter, and helping ensure that every software release is as secure and reliable as possible without sacrificing developer velocity.

1.6 Scope and Limitations

1.6.1 Scope of the Research

This study builds and tests a smart, self-learning malware detection system for cloud-based software development pipelines (CI/CD). Here's a simpler breakdown:

1. It uses a neural network that can adapt itself in real time to spot new, unseen (zero-day) threats.

2. The system runs continuously within CI/CD pipelines, helping developers catch malware early in the development process.

3. Its defenses are improved using automated adversarial mutation strategies during training, especially focusing on malware that keeps changing (polymorphic and metamorphic).

4. The research measures how well it performs in terms of accuracy, false positives, speed (latency), and adaptability, comparing it with existing threat detection models from literature.

5. It does not focus on areas like building full enterprise products, legal or compliance issues, low-level network security, or human factors — the main emphasis is on technical design and testing.

1.6.2 Research Challenges

CI/CD environments change very quickly, with rapid iterations and continuous deployments of executable artifacts. Because of this constant movement, it's hard to gather and analyze binary data in a consistent way without halting developer pipelines. Rapidly evolving malware risks model overfitting and catastrophic forgetting, so architectures must adapt while remaining stable. Labeled polymorphic threat data are scarce, often requiring synthetic adversarial mutation datasets that must mirror real-world threats. Increasing sensitivity to catch novel attacks can raise false positives and disrupt deployments. Deep-learning integration must preserve throughput by minimizing latency and resource use. The detector itself can be attacked, so adversarial robustness and model integrity are essential. Real-time processing of large artifact streams demands computational efficiency and scalable infrastructure.

1.7 Team Overview

The research was conducted collaboratively by a group of four final-year undergraduate students from the Department of Computer Science and Engineering at BRAC University. Every member provided unique skills and knowledge related to the technological aspects of the project. The proposed adaptive threat should be designed, implemented, and evaluated by domains. Detection framework. Table 1.1 sets out the composition of the team and main responsibilities. The project was supervised by Mr. Partha Bhoumik, Lecturer, Department of Computer Science and Engineering, BRAC University, who had been providing consistent support

and guidelines. The project was supported by Computer Science and Engineering Department, BRAC University, with advice on the direction of research, technical feasibility, and the evaluation method. All four members were involved in the design of the threat-mutation adversarial together. Training loop, weekly progress checks, and preparation for this thesis document. Despite all the above, some architectural decisions, such as selecting a feature fusion approach as concatenation, still leave room for improvements. Consensus was reached after empirical data evidence on the role of overattention mechanisms was collected and evaluated by the team.

Name	**Student ID**	**Primary Contribution**
Sahil Ishan Tonmoy	22301612	BiLSTM branch development for temporal API-call sequence analysis; adversarial mutation engine design; continuous learning module implementation and model-evolution logic; integration of the CI/CD pipeline with GitHub Actions.
S.M. Mashrafi Zaman	22301614	STRIDE threat modeling, CNN branch development for spatial (binary-to-image) malware analysis, and coordination of the overall implementation workflow.
Shafayet Karim	22101507	Literature review; feature fusion layer experimentation, and comparative evaluation of concatenation, addition, and attention strategies.
Shifat Ahmed	22301582	System architecture design, dataset preprocessing, performance benchmarking against baseline threat detection models, and documentation.

Table 1.1: Team Members and Primary Contributions

1.8 Key Learnings and Insights

The challenges encountered by the team in working on this research gave them a variety of technical and collaborative knowledge that went beyond what was taught in the regular school courses. The following is a summary of the most important learning that took place during the The design, implement and evaluation of the project.

Static analysis alone is fundamentally insufficient for modern threats. The literature review was one of the most important and reinforcing lessons. An almost static signature-based detection is always a possibility for all of the papers surveyed. consistent technique. Resisted polymorphic metamorphic malware. This authen- ticated the heart of this. The motivation of the research was the influence on all subsequent architectural decisions. In particular, with respect to the use of a CNN spatial The model consists of a branch and a BiLSTM. On both their temporal branches, neither.

Behavioral analysis through API call sequences is highly discriminative. The best results were obtained with the Malware Bazaar dataset and training the BiLSTM branch. All the models described above have been performed. The team has an understanding of how malware is executed at runtime. It is the behavior and not its static structure. The order of the API calls is in a sequence. sequences and patterns that have a meaning for which models can be learned, in the case of a car, Convolutional Neural Network because it was easier to train the former than the latter. A standard This was the reason that CNN was easier to train than a bidirectional LSTM. Thank you to the son for his choice of bidirectional LSTM. In a unidirectional model, both previous and upcoming API calls are involved in the context. The effect of the feature fusion strategy on Performance.

Feature fusion strategy has a measurable and non-trivial impact on performance. The three fusion experiments are done by running the three concatenation, addition, and attention experiments (concatenation, addition, and attention). One of the most educational experiments of the project was the experiment on attention. That such a simple concatenation (99.00% accuracy) beat the more complicated one. The very first model that was used, a complicated attention mechanism (94.58%), was unexpected at first sight. It highlighted that it is important to capture all of the information from both sides independently. It was not the correct method to weigh them against one another; instead, it was the proper way to do this. scale and a particular problem/dataset.

Adversarial training generalizes beyond the specific attack it trains on. The main lesson learned from the mutation strategy experiments was that the random insertion of the gene into the genome was the most critical factor. The strategy, which is trained to resist insertion-type noise, also gained 98.67% robustness against replacement-type attacks. It was demonstrated that this cross-protection effect is that the adversarial perturbation could be implicitly taught through the training against one form of perturbation. Model to ignore a wider class of noise, which is a practically important property for any realistic deployment in which the exact attack type is not known in advance.

Continuous learning requires a safety gate. The implementation of the model evolution loop showed that it is not enough to blindly retrain and deploy an updated model. can cause performance degradation. The choice of decisions to compare each retrained model against the production baseline before promoting it and to A key design protection that was discarded was models that don't improve. This mirror was a meaningful best practice from production machine learning systems' real-world engineering lessons.

Security and pipeline velocity are not mutually exclusive. A recurring con-Concern During the design process, the concern was whether the deep learning inference May be directly inserted. The latency of the CI/CD pipeline would be too high. The team has learned that the retraining was sped up by a GPU, and the im-Implementation supports mixed-precision arithmetic. The average scan cycle can be maintained at and below the confidence threshold, as in the case of fast-pathing. The well-designed security layer does, within milliseconds. It does not need to be a limiting factor.

Modular architecture enables iterative development. Building the system As six separate modules, each having its own input and output and responsibility, made it possible to develop, test, and debug each component in isolation before

integration. This made debugging easier and enabled The importance of having parallel development across team members is an important practical lesson. In Research systems in software engineering.

Chapter 2

Literature Review

2.1 Preliminaries

2.1.1 Adaptive Evolving Neural Security Architectures

Adaptive evolving neural security architectures refer to intelligent systems that leverage neural networks capable of learning and self-improvement over time. These architectures continuously analyze new data and threat patterns, allowing them to autonomously adjust their defensive strategies. This dynamic approach is particularly effective in environments where cyber threats are constantly changing, such as cloud and IoT ecosystems.

2.1.2 Cloud-Native CI/CD Security and Real-Time Defense

Cloud-native CI/CD security integrates protective measures directly into the software development and deployment pipeline. By embedding security checks and automated defenses within continuous integration and continuous deployment (CI/CD) processes, organizations can identify and address vulnerabilities as code is developed, tested, and released. Real-time defense systems help keep applications safe while they're running, making sure security keeps up with the fast and scalable nature of cloud environments.

2.1.3 Polymorphic and Metamorphic Malware Detection

Polymorphic malware keeps its payload the same but changes its external form, while Metamorphic malware rewrites its own code through each generation to change the payload. Detection techniques that focus on behavioral analysis and self-learning techniques can recognize these threats better than solely relying on static signatures

2.1.4 Behavioral and Code-Level Anomaly Detection

Behavioral detection techniques involve differentiating the normal runtime behaviour and activities that deviate from the learned baseline from normal runtimes, while Code-level anomaly detection inspects the structure and instructions of the code to spot abnormalities.

2.1.5 Static and Dynamic Threat Analysis (SAST/DAST)

SAST involves analyzing without running the app to find vulnerabilities in code, bytecode, or binaries, while DAST probes a running application from outside to find exploitable flaws.

2.1.6 Adversarial Training

Adversarial, in a general sense, means introducing an opposition that tries to fool the system. Adversarial training is a method where models are trained by inputting intentionally perturbed samples to make the model predict wrong and learn decision boundaries

2.1.7 Red Team Simulation

Red team simulation is a security exercise where authorized attackers emulate real adversaries to test how well a system performs its security procedures and its detection technology.

2.2 Review of Existing Research

To systematically organize the literature, twenty research papers were reviewed and grouped into five key themes based on their focus, methods, and relevance to the proposed adaptive threat detection framework. This structure enables clear comparison across studies that address different aspects of cloud CI/CD security and intelligent defense. The themes include: (1) Adaptive AI and Neural Architecture Design, exploring self-evolving neural models; (2) Cloud and CI/CD Pipeline Security, centered on protecting DevOps workflows and automated releases; (3) Threat Characterization and Detection, analyzing malware behaviors and detection methods; (4) Analytical and Testing Methodologies, focused on SAST and DAST-driven vulnerability assessments; and (5) Adversarial and Simulation-Based Defense, investigating Red Teaming and adversarial learning techniques. Organizing the research in this way helps reveal how each area contributes to AI-driven cloud security and exposes the critical gaps this study seeks to address.

2.2.1 Adaptive AI and Neural Architecture Design

The study aims to highlight the advantages of ENN, Evolving Neural Networks, for decision-making for effective discovery and adaptive behaviors across complex domains. They employed Neuroevolution techniques to optimize network parameters and architectures simultaneously but separately. They used CNE focusing on evolving connection weights of fixed topology networks through genetic evolution and Advanced Neuroevolution for individual neurons, partial networks, and evolving network topology simultaneously with NEAT. They also used Indirect Encoding to efficiently represent large networks with reusable structure, allowing scaling to millions of connections. The combined Neuroevolution with supervised learning algorithms to fine-tune network weights. Through their combined system, they

found that it outperforms traditional reinforced learning approaches by a magnitude in complex control problems. They found that Neuevolution demonstrates strong adaptability and robustness in dynamic and non-Markovian environments, especially when combined with supervised learning, which further enhances adaptability(**Miikkulainen & Stanley, 2009**)[5].

The research focuses on evolving ANNs for adaptive behavior in artificial agents, arguing that evolving both the architecture and learning parameters incrementally allows for the emergence of a robust, noise-resistant network that is capable of complex, adaptive behaviors in dynamic environments. This study models recurrent neural networks with excitation and inhibition channels, nonlinear transfer functions, veto mechanisms, and internal noise to emulate biological neural variability to enhance adaptability. They use an extended genetic algorithm, SA-GA, allowing both connectivity and network size to grow as needed. They start with minimal random changes in connected network populations, increasing complexity only when beneficial to fitness, measured by a robot's ability to navigate to and stay centered in an arena using limited sensors. The metrics and analysis reveal that this evolutionary process promotes modularity and the emergence of a biologically inspired adaptive control system(**Harvey & Husbands, 1992**)[2].

Another study explores the use of Neural Architecture Search (NAS) to discover inherently robust neural network architectures that can resist adversarial attacks. This research proposes an evolutionary NAS approach, Robust Neural Architecture Search (R-NAS), which simultaneously optimizes for accuracy and robustness against adversarial attacks using a combined fitness function. The neural architectures are evolved across layers, blocks, and model level by diverse mutation operations and spectrum-based niching to maintain population diversity. By testing each design against black-box attacks on the CIFAR-10 dataset, it balances high performance on normal data with strong resistance to adversarial interference. R-NAS created neural networks that stayed highly accurate while being more than twice as resistant to adversarial attacks compared to traditional methods(**Kotyan & Vargas, 2020**)[14].

This investigation introduces an adaptive neural network model with reflection capable of autonomously evolving its internal structure and learning parameters in response to dynamic environments, modeling how biological neural networks evolve and adapt both their architecture and behavior, overcoming the limitations of fixed-topology neural networks. It introduces neuro-agents, modular units made up of smaller components called neuro-objects that learn together by sharing information and working in parallel. These modules can be organized hierarchically, combining similar and diverse learning parts to form large, flexible networks that adapt and work together seamlessly on complex problems. The study shows that neural networks can grow and adapt both their design and learning at once, combining specialized parts into large systems that work together without losing their ability to learn(**Tsukamoto & Namatame, 1996**)[1].

2.2.2 Cloud and CI/CD Pipeline Security

This paper used the CICD2019 dataset to detect threats. It encoded the CICD logs to an Autoencoder (AE) so that it can feed insightful features of this dataset to a Multi-layer Perception (MLP) Deep Learning model that can detect threats. Then the threats were categorized into five categories.

Normal: expected network behavior.
DoS (Denial of Service): Floods the network with traffic
Probe: Scanning aimed at discovering vulnerabilities.
U2R (User to Root): Attempt to gain access to root by using a privileged user like admin.
R2L (Remote to Local): Remotely making unauthorized attempts to gain local network access.
(**Modalavalasa & Yadav, 2025**)[23]

This paper uses Docker image scanning tools to detect malicious activity. Uses Snyk and StackHawk to detect malicious activity throughout the application development cycle. This paper combines two technologies to improve CICD security. They proposed both SAST and DAST type testing for more secure pipeline security.

1. Snyk's focus on SAST (Static Application Security Testing)
2. StackHawk's DAST (Dynamic Application Security Testing)

(**Marandi, Bertia, & Silas, 2023**)[17]

In this paper, when pushing code in GitHub, it automatically triggered Jenkins, then it fetched the code, ran OWASP Dependency Checker, SonarQube, and Trivy to ensure code quality and check vulnerabilities. Then it was pushed to a Docker image and deployed through AgroCD. AgroCD checks configuration requirements before deployment. AWS VPC isolates sensitive data in different subnets to avoid public exposure. Monitoring was done by Prometheus and CloudWatch. They provided real-time detection of security issues across the pipeline (**Ashtagi et al., 2025**)[21].

This paper broadly focuses on how software development can be modernized with the integration of governance and risk management. From development cycles to deployments and integrations, it proposes a unified framework combining SDLC, CI/CD, and DevOps. For security, it includes Security methods, including SAST, DAST, SCA, Threat Modeling, Risk-Based Testing, and penetration testing (**Ganugapati & Prabhu, 2023**)[16].

2.2.3 Threat Characterization and Detection

The research introduces a novel malware detection system that addresses the shortcomings associated with existing signature-based scanners, which are susceptible to typical obfuscation techniques such as code reordering and register renaming. It uses instruction sequence templates to represent malware behaviors and employs

an intermediate representation of machine instructions for architecture-independent analysis. A semantics-aware detecting algorithm performs symbolic execution and theorem proving for verification of semantic equivalence while avoiding def-use chains corruption. The results of the experiment demonstrate that using this approach, it is possible to detect several worm variants using far fewer signatures and without any false positives on benign software, thus substantially surpassing standard static analyzers when it comes to tackling obfuscated code. Nevertheless, the proposed approach has some limitations: first, they should be successfully disassembled in order to analyze their instructions; second, there are supported instruction forms; and lastly, there are strict matching rules. This may cause a mis-catch for some variations. However, all these factors lead us to concentrate on Windows IA-32 binaries (**Christodorescu, Jha, Seshia, Song, & Bryant, 2005**)[3].

This study introduces a deep learning method that combines convolutional and recurrent neural networks to classify malware on the basis of system call sequences, which are collected at runtime in a sandbox. A simple list of API calls can be converted to their corresponding scalar representations; it reveals the hierarchical structure of API calls in software and improves the accuracy for discriminating malware families. When tested on a balanced dataset created from antivirus signatures, the ConvNet-LSTM model showed over 89% accuracy, which is a better performance than other machine learning models. Nonetheless, this model only considers system calls, may have imprecise labels and suffers from possible high computational requirements of dynamic analysis as well as evasion attacks (**Kolosnjaji, Zarras, Webster, & Eckert, 2016**)[7].

Another investigation holds the core information about the methods that have been developed by malware over a period of 10 years to evade dynamic tools like sandboxes and debugging software. Based on this, it enlists 92 different techniques in 16 groups and surveys more than 45k malware samples from the time frame of 2010–2019 against benign software. The results demonstrate that approximately 80% of malware uses evasions, where the most common and evolving types are timing and stalling, in response to more advanced sandboxing techniques. Evasive methods show high correlations with some malware families or packers. Despite continual technique development, there was only a modest increase in the use of evasive techniques throughout the study period. These include limiting consideration to 32-bit binaries only, the possibility of analysis tool detection by malware, and omissions regarding long-delayed behaviors' evasions (**Galloro, Polino, Carminati, Continella, & Zanero, 2022**)[15].

This work systematically evaluates malware detection techniques based on static analysis, dynamic analysis, and hybrid variants by training Hidden Markov Models (HMMs) on opcode and API call sequences extracted with IDA Pro (static) and Buster Sandbox Analyzer (dynamic). Using seven malware families (e.g., Zbot, ZeroAccess, Harebot, Winwebsec, Security Shield, Smart HDD) and benign Windows System32 files, the study compares four train/score settings—static-static, dynamic-dynamic, dynamic-static, and static-dynamic—under 5-fold cross-validation with ROC-AUC and Precision-Recall metrics. Results show that dynamic training and scoring deliver the strongest detection performance overall, with static-static often

competitive for API calls but typically behind dynamic; simple hybrids yield mixed outcomes, where dynamic-static underperforms both pure approaches and static-dynamic performs worst. API call sequences outperform opcode sequences, likely due to opcode brittleness under obfuscation. The authors note limitations including reliance on a single sandbox/disassembly toolchain, potential labeling noise in family assignments, class imbalance affecting PR more than ROC-AUC, and uncertain generalizability beyond the tested families and Windows setting (**Damodaran, Di Troia, Corrado, Austin, & Stamp, 2022**)[9].

This research introduces a behavioral malware detection approach using Deep Graph Convolutional Neural Networks (GCNs) to analyze API call sequences. The study converts API call sequences into graph structures where nodes represent API calls and edges capture sequential dependencies. By applying graph convolutional layers, the model learns hierarchical representations of malware behavior patterns. Evaluated on a dataset of 42,797 malware and 1,079 goodware API call sequences extracted from Cuckoo Sandbox reports, the graph-based representation proved effective at capturing complex behavioral patterns that traditional sequence-based methods might miss. The authors note limitations including the imbalanced dataset and the computational overhead of graph construction (**Oliveira & Sassi, 2019**)[12].

Another study developed a deep learning-based sequential model for malware analysis using Windows API calls from executable files. The research utilized the Mal-API-2019 dataset containing 7,107 malware samples across eight families (Spyware, Downloader, Trojan, Worms, Adware, Dropper, Virus, Backdoor) with 342 distinct API calls. The model employed Long Short-Term Memory (LSTM) networks to capture temporal dependencies in API call sequences, treating malware detection as a sequence classification problem. The LSTM-based approach demonstrated the effectiveness of recurrent neural networks for learning from behavioral patterns in API call sequences, highlighting that sequence-based deep learning models can effectively distinguish malware families based on their runtime behavior (**Catak, Yazi, Elezaj, & Ahmed, 2020**)[13].

This work presents a malware classification approach using deep convolutional neural networks (CNNs) applied to malware binaries converted into grayscale images. The study transforms raw executable files into visual representations, allowing CNNs to extract spatial features and patterns characteristic of different malware families. The proposed M-CNN architecture was evaluated on two benchmark datasets: the Malimg dataset (9,339 malware images across 25 families) and the Microsoft Malware Classification Challenge dataset. The image-based approach proved highly effective for static malware analysis, as visual patterns in binary code remain relatively stable even when malware employs basic obfuscation techniques. However, the method's reliance on static features makes it vulnerable to advanced packing and encryption techniques (**Kalash, Rochan, Mohammed, Bruce, Wang, & Iqbal, 2018**)[10].

2.2.4 Analytical and Testing Methodologies (SAST/DAST)

The study focuses on Android malware detection modeled through API call behavior using Markov Chains. Using a virtual environment, they ran the API calls and stored the logs. For automated input, they used the Monkey tool, which can also be useful in our concept. The approach used both static and dynamic features to analyze system behavior in different environments. They used a Random Forest Classifier to categorize the features (**Onwuzurike et al., 2018**)[11].

This research focused on detecting malware in real time using a deep learning approach. It developed a dual-branch model where one branch analyzed static features, while the other looked at dynamic features. The results from both branches were combined and processed through deep neural network layers. The DNN was trained on 5.4 million PE samples to learn effective pattern representations(**Mihalache et al., 2024**)[19].

Another study focused on automating DevSecOps pipelines by integrating AI. It used application testing to detect attacks and combined SAST for finding code-level vulnerabilities with DAST to catch runtime attacks like SQL injection. The system also leveraged AI-driven frameworks such as GDPR, HIPAA, OWASP, and NIST to check compliance and make sure security standards were met.This study looks at the ongoing battle between malware trying to evade detection and machine learning defenses trying to stop it. Researchers created a model that teaches malware to trick detection systems, and a defense system that trains classifiers to be tougher and more reliable against such tricks(**Abdiukov, 2024**)[18].

This study tackles the common problem of false alarms in detection models, which can lead to mistakes. To make the results more accurate, the researchers added an active behavioral validation step to Cloud Security Posture Management. This step simulates real attacks to test and verify AWS components like S3, EC2, and IAM settings, helping improve the overall accuracy of threat detection (**Dikshant & Verma, 2025**)[22].

2.2.5 Adversarial and Simulation-Based Defense

This study looks at how hackers and security systems are constantly trying to outsmart each other. The researchers build a system that shows how malware can fool machine learning–based detectors, then create another system that helps those detectors become smarter and harder to trick. This model uses API calls from PE files, simulates attacks via feature manipulation, and defends via adversarial training with security regularization. Evasion attack model achieved strong evasion with small feature changes, Secure-learning defense restored high detection accuracy, and outperformed commercial scanners. This work provides a strong foundation for building resilient, security-aware malware detection systems by successfully demonstrating that incorporating adversarial training through SecDefender can significantly strengthen malware classifiers against sophisticated evasion attacks (**Chen, Ye, & Bourlai, 2017**)[8].

This research aims to bridge the gap between AI theory and real-world cybersecurity applications by reviewing and analyzing modern cybersecurity frameworks that claim to use AI, ML, or DL for threat detection, prevention, and response. From 3,200 vendors, 38 cybersecurity frameworks were identified, focusing on IDPS, EDR, and IAM systems, and they were classified into groups — AI-powered, ML-powered, DL-powered, rules-based, or unknown/no info. Most cybersecurity frameworks are still rules-based or traditional ML-driven; few fully use deep learning, where only NVIDIA Morpheus stands out for its multi-GPU AI pipeline. Overall, the paper provides a comprehensive assessment of AI and deep learning integration within modern cybersecurity frameworks, revealing a significant gap between theoretical advances and practical deployment (**Applebaum, Miller, Strom, Korban, & Wolf, 2016**)[6].

Another investigation centers on black-box targeted adversarial attacks by generating more transferable adversarial samples, which proposes a new robust training framework that adjusts the decision boundaries of surrogate models to improve attack transferability. The approach is a generative adversarial framework consisting of adversarial samples from clean images, where a U-Net architecture is used, known as an attack generator, and controlled noise to smooth the surrogate model's decision boundaries via robust training, known as a perturbator. The Discriminator (D) is robustly trained to reduce discrepancies between the surrogate and target models, improving sample transferability and Experiments are performed on ImageNet using pretrained CNNs as both surrogate and target models. Overall, the paper presents an effective and computationally efficient framework for generating transferable adversarial examples through robustly trained surrogate models, and the method significantly enhances attack success rates against black-box targets while maintaining fast generation speed (**Shi, Wang, Li, Guo, & Liu, 2024**)[20].

This work describes a Red Team/Blue Team exercise designed to test a collaborative DDoS defense system combining two mechanisms, COSSACK and D-WARD, by evaluating how well the system detects and mitigates different types of DDoS attacks, identifying vulnerabilities and weaknesses in both systems, and gathering insights for improving real-world collaborative DDoS defenses. Here blue team is the developers of COSSACK and D-WARD (defenders), the red team is the attackers from Sandia Labs, who designed and executed diverse DDoS attacks, and the white team is the Neutral coordinators from BBN Technologies, responsible for data collection and analysis. Here, nine connected edge networks are linked through a simulated core router, and realistic legitimate traffic is generated using the Skaion system. The combined COSSACK–D-WARD defense maintained high legitimate traffic and low false positives during spoofing and rolling floods, though timing-based floods caused brief disruptions. In contrast, ACK Proxy attacks effectively bypassed detection, severely degrading performance. Overall, the study provides one of the first realistic evaluations of collaborative DDoS defenses under controlled Red Team/Blue Team conditions, where the results show that COSSACK and D-WARD can effectively mitigate common flooding attacks but remain vulnerable to sophisticated proxy-based and timing attacks (**Mirkovic et al., 2008**)[4].

2.3 Summary of Key Findings

Table 2.1: Strategies and features found in papers

Research Papers	**Key Strategy**	**Feature**
Modalavalasa and Yadav [23]	AE-MLP Model (Autoencoder & Multi-layer Perception)	AE takes the dataset, encodes insightful features, and feeds them to MLP.
Marandi et al. [17]	Static Application Security Testing (SAST)	Uses Snyk to detect vulnerabilities during development.
Marandi et al. [17]	Dynamic Application Security Testing (DAST)	Uses StackHawk to test and secure the running application/pipeline.
Ashtagi et al. [21]	Software Composition Analysis (SCA)	Uses OWASP Dependency-Check to scan third-party libraries for known vulnerabilities.
Ashtagi et al. [21]	Container Image Scanning	Uses Trivy to scan Docker images for vulnerabilities and misconfigurations.
Ashtagi et al. [21]	Cloud-Native Architecture	Uses AWS VPC, EC2, CloudFront, and CloudWatch for secure, scalable, resilient deployments.
Ashtagi et al. [21] Ganugapati and Prabhu [16]	Infrastructure as Code (IaC)	Uses Terraform to provision and manage AWS infrastructure.
Miikkulainen and Stanley [5] Harvey et al. [2]	Evolving Neural Networks (CNE/NEAT, indirect encoding)	Evolves topology and/or weights; robust in non-stationary tasks; indirect encoding scales to large networks.
Kotyan and Vargas [14]	Robust NAS (R-NAS)	NAS jointly optimizes accuracy and adversarial robustness; improves resistance to black-box attacks.

Continued on next page

Research Papers	Key Strategy	Feature
Tsukamoto and Namatame [1]	Adaptive neural model with "neuro-agents"	Modular, hierarchical neuro-objects evolve structure and learning parameters in parallel.
Christodorescu et al. [3]	Semantics-aware instruction-template detection	Architecture-independent IR + symbolic execution/theorem proving; fewer signatures, low false positives.
Kolosnjaji et al. [7]	ConvNet-LSTM on system-call sequences	Dynamic, sequence-based malware classification from sandboxed API calls.
Galloro et al. [15]	Evasion tactics landscape	Widespread sandbox/debugger evasion (e.g., timing, stalling); need behaviorally robust defenses.
Damodaran et al. [9]	Static vs. dynamic vs. hybrid detection	Dynamic analysis generally outperforms static; API-call features often beat opcode features.
Oliveira and Sassi [12]	Graph CNN on API call sequences	Converts API sequences to graphs; GCN learns behavioral patterns from imbalanced dataset.
Catak et al. [13]	LSTM on API call sequences (Mal-API-2019)	Sequential deep learning on malware samples; effective for temporal behavior patterns.
Kalash et al. [10]	M-CNN on malware images (Malimg)	Converts binaries to grayscale images; CNN extracts visual patterns for classification.
Chen et al. [8]	Adversarial training (SecDefender)	Security-aware training increases robustness to feature-manipulation/evasion attacks.
Applebaum et al. [6]	State of AI in cyber products	Many frameworks remain rules/ML-based; limited DL adoption; motivates research gap.
Shi et al. [20]	Transferable adversarial examples (black-box)	Robustly trained surrogate models (generator/perturbator/discriminator) yield transferable attacks.

Continued on next page

Research Papers	**Key Strategy**	**Feature**
Mirkovic et al. [4]	Red/Blue-team evaluation (COSSACK & D-WARD)	Effective vs. spoofing/rolling floods; gaps vs. ACK-proxy/timing attacks.

Chapter 3

Requirements, Impacts and Constraintsn

3.1 Project Management Plan

The project was executed across the Spring 2026 term in four incremental phases:

1. **Phase 1 — Research and Literature Review:** Survey of 20 Reviewed paper on five thematic areas, gap analysis and problems; formalized the reviewed paper; and presented the reviewed paper.

2. **Phase 2 — System Design:** Specification of the architecture of the six-module pipeline, the threat modeling of the STRIDE threats, and the selection of the technology stack.

3. **Phase 3 — Implementation:** the CNN and BiLSTM layers were created. The CNN and BiLSTM layers were developed. For the branches, a feature The following classification heads were used: fusion layer, an MLP classification head, and adversarial mutations. The branches, feature fusion layer, MLP classification head, and adversarial The following were used: mutation, action engine and continuous learning loop.

4. **Phase 4 — Evaluation and Documentation:** students evaluated and documented their work. In Phase 4, Experimental Benchmark- students evaluated and documented their work. ing: Fusion strategies and mutation strategies. Analysis on baseline threat detection models; thesis writing.

3.2 Risk Management

The following are key risks identified and mitigation strategies:
Data scarcity: Few labeled data sets of malware exist. Mitigation: Use the Malware Bazaar public datasets and synthetic samples of the Malimg public datasets. Developed with the adversarial mutation engine.
Model overfitting: Deep learning models can be used to fit the training set. distributions. Regularization: L2 Regularization, Dropout, Momentum, Weight Decay, Gradient Decay and LROnPlateau Adversarial training with various mutation strategies and schedulers.

Adversarial attacks on the detector: The detection model can be attacked. Mitigation: Randomly insert an adversarial training strategy, model Continuous retraining using new samples and integrity checks.
Latency in pipeline: Deep learning inference can cause a delay in de- ployments. Training: Mixed-precision training with the same device type; safety; Using thresholding, fast-pathing low-risk artifacts.

3.3 System Specification

3.3.1 Threat model

STRIDE Category	Threat Description	Affected Components
S - Spoofing	Attacker impersonates legitimate developer or CI/CD service to inject malicious code	Authentication Layer, Git Repository, Pipeline Triggers
T - Tampering	Modification of training data, model weights, or detection thresholds to evade detection	Model Storage, Training Dataset, Configuration Files
R - Repudiation	Attacker denies malicious code submission or claims false positives	Audit Logs, Detection History, Version Control
I - Information Disclosure	Exposure of model architecture, detection patterns, or sensitive code artifacts	Model Parameters, API Endpoints, Scan Results
D - Denial of Service	Overwhelming the detection system with requests or adversarial samples to bypass scanning	CNN/BiLSTM Inference Engine, Pipeline Queue, Resource Limits
E - Elevation of Privilege	Exploiting vulnerabilities to gain admin access and disable security checks	Access Control, Deployment Gates, Admin Console

Figure 3.1: STRIDE Threat Model

To formally define the adversarial landscape that the Adaptive Threat Detection framework must mitigate, we utilized the STRIDE threat modeling methodology (Figure 3.1). Developed by Microsoft, STRIDE systematically categorizes potential system vulnerabilities into six distinct attack vectors. In the context of our automated CI/CD pipeline and deep learning architecture, these vectors manifest as

follows:

- **Spoofing:** An adversary may attempt to bypass the local Git pre-commit hooks by impersonating a trusted developer or a privileged CI/CD service account to push malicious binaries upstream.
- **Tampering:** The Continuous Adaptation Engine relies on dynamically retraining the PyTorch model weights. Attackers may attempt data poisoning by modifying the quarantined datasets or altering the CI/CD workflow configurations to create detection blind spots.
- **Repudiation:** Without immutable logging, a compromised developer account could push obfuscated malware and subsequently deny the action. The framework mitigates this by enforcing strict version control and automated audit trails for every intercepted artifact.
- **Information Disclosure:** Exposure of the BiLSTM's sequence vocabulary or the CNN's feature extraction weights would allow an attacker to craft perfect "white-box" adversarial mutations specifically designed to bypass the detection boundaries.
- **Denial of Service (DoS):** The dual-branch inference engine operates locally. Malicious actors could flood the pipeline with massive, heavily obfuscated PE files to stall the local CPU or overwhelm the asynchronous GitHub Actions retraining queue.
- **Elevation of Privilege:** An attacker might exploit external vulnerabilities to gain administrative access to the repository, thereby disabling the security deployment gates and bypassing the AI-driven code review entirely.

By explicitly defining these threat vectors, the architectural design outlined in subsequent chapters—specifically the integration of adversarial mutation training and isolated local inference—was engineered to ensure robust MLSecOps resilience against each STRIDE category.

3.4 Societal Impact

Production environment impacts. The use of adaptive AI security in CI/CD pipelines has real-world consequences. societal implications. As software delivery enables services for critical infrastructure, such as healthcare systems, financial infrastructure and more. public utilities, The safety of deployment pipelines directly impacts public safety and trust. By discovering threats at an earlier stage in the development lifecycle and automat- ing remediation, the proposed framework reduces the risk of large-scale data breaches. and service disruptions are minimized. Outages that can impact end users. For or- ganisations in developing economies such as In Bangladesh, the adoption of cloud While computing is on the rise, skills in cybersecurity are still lagging behind. is still low and an automatic and learning detection system lowers the barrier to ensure the deployment of software. Moreover, the system reduces false positives, thus minimizing unnecessary Reducing Operational interruptions leads to improved developer productivity. Playing a role in the speed of delivery of the beneficial digital services.

3.5 Ethical Issues

There are a number of challenges to overcome in creating and deploying an AI-powered strategy. There are some problems that should be solved in making and applying an AI-driven method. security system: ethical responsibilities. The fake threat mutation module is created. A fake threat mutation module is created. malware. Samples for training purposes. Only these samples can be held by only trained personnel and not deployed or distributed outside of the training environment! Under experimental conditions. Misuse of created adversarial samples Would pose a threat if they were put into the hands of bad guys. Also, the system gathers and analyzes pipeline artifacts and code commits. Sensitive intellectual property may be included in multiple API calls, and the sequence of these calls can be determined. Access data should be subject to: controls, anonymization of data, and audit trails should be enforced, and the data should be: Security monitoring does not impact developers' privacy or organizational con- fidentiality. For instance, to block deployments or to determine eligibility for buying neuroprosthetics. The risk of revoking a credential is due to potential misdeeds. Transparency in model These are important safeguards for decisions and for provision for a human review override.

Chapter 4

System Design and Implementation

4.1 Methodology Overview

The proposed threat detection framework was developed iteratively using a modular, defense-in-depth architecture designed for seamless integration into Continuous Integration and Continuous Deployment (CI/CD) pipelines. The system is conceptually divided into four fundamental stages: Input and Interception, Multi-Modal Artifact Parsing, Dual-Branch Threat Detection, and Action & Continuous Learning.

At the core of the methodology is a hybrid deep learning architecture that evaluates artifacts simultaneously across two modalities. First, a static analysis branch utilizes a Convolutional Neural Network (CNN) to evaluate the structural and spatial layout of Portable Executable (PE) binaries transformed into 2D image representations. Second, a dynamic analysis branch leverages a Bidirectional Long Short-Term Memory (BiLSTM) network to trace sequential API execution behaviors, capturing runtime anomalies that static analysis might miss. The high-dimensional feature vectors from both branches are subsequently merged using a concatenation-based Feature Fusion layer, which passes the aggregated representation to a dense classification network to generate a final threat verdict.

To ensure resilience against sophisticated evasion techniques, the model undergoes adversarial hardening during the training phase. By applying stochastic mutation strategies—such as random sequence insertion, replacement, and noise appending—the behavioral sequence branch is trained to maintain high detection efficacy even when confronting zero-day obfuscation tactics. Finally, the system enacts immediate automated remediation via a dedicated Git interceptor; upon detecting malicious artifacts, the system dynamically quarantines the offending files, blocks the CI/CD pipeline, and cancels the commit job to prevent upstream contamination. This highly automated, self-contained architecture provides robust, near real-time protection while maintaining the high-throughput performance required by modern DevOps environments. An overview of the full research pipeline is shown in Figure 4.1.

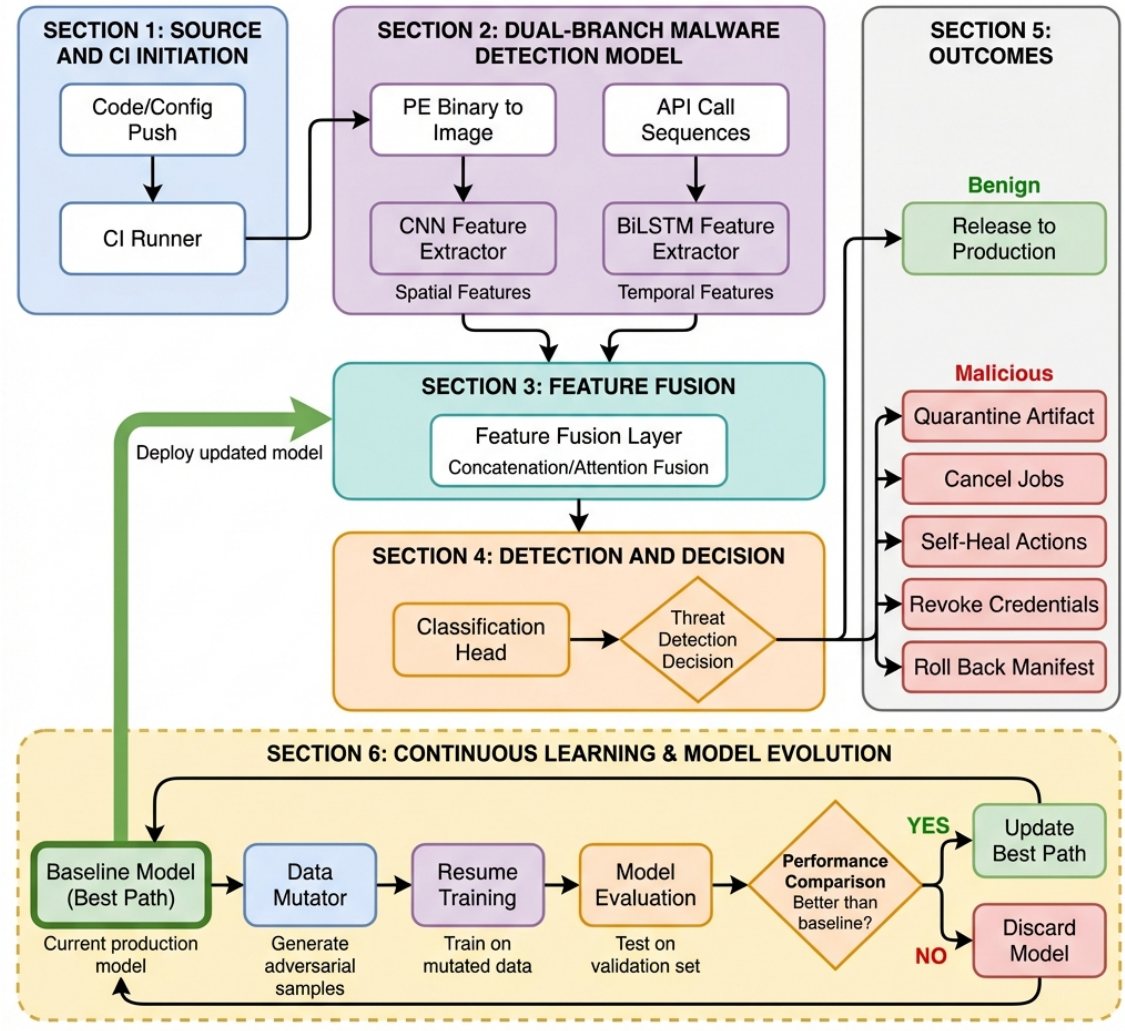


Figure 4.1: Methodology Diagram

4.2 Data Collection

The dataset utilized in this research integrates static and dynamic feature representations. The samples were collected and compiled from two distinct sources to form a balanced set of malware and benign binaries:

1. **Malware Samples (Static & Dynamic):** Procured from **Malware Bazaar**, a public repository of verified malware samples. The malware cohort consists of 2,000 unique Windows Portable Executable (PE) binaries. For each sample, both the raw executable byte streams (for the static CNN branch) and sandboxed API call traces (for the dynamic LSTM branch) were retrieved and aligned.

2. **Benign Samples (Static & Dynamic):** Procured locally by scanning and extracting system files and applications from a standard Windows environment across three main directories:

   - `C:\Windows\System32` (system executables and libraries)
   - `C:\ProgramFiles` (user applications)

- `C:\ProgramFiles(x86)` (32-bit user applications)

To prevent the model from developing classification bias toward specific binary formats, the collection process was configured to balance the distribution of **Executables (`.exe`)** and **Dynamic Link Libraries (`.dll`)**. A balanced quota of `.exe` and `.dll` files was extracted recursively from the target directories. Static PE images were generated from the raw binary files, and dynamic API sequences were extracted statically from the Import Address Table (IAT) using the PE import directory. A total of $1,998$ benign samples were scanned and incorporated.

4.2.1 Data Cleaning

To ensure quality and consistency across branches, the following cleaning steps were executed:

- **Corrupt File Filtering:** Binary samples with corrupted headers or incomplete byte-streams that failed to produce valid image representations were removed from the cohort.
- **Empty Sequence Removal:** API execution traces that contained zero or purely repetitive null calls were discarded.
- **Token Sanitization:** API strings were cleaned of system-specific memory addresses and parameters to prevent vocabulary bloat, focusing strictly on standard API call identifiers (e.g., `RegOpenKeyExW`, `VirtualAlloc`).

4.2.2 Data Transformation

Feature branches required distinct mathematical transformations before model feeding:

- **Static PE Images:** Raw 1D byte arrays were converted to 2D representations. The resulting matrix was resized to a uniform 128×128 dimension using bilinear interpolation. Pixel intensities were normalized to $[0.0, 1.0]$:

$$x_{\text{static}}^{(i)} = \frac{x_{\text{pixel}}^{(i)}}{255.0} \tag{4.1}$$

- **Dynamic Sequences:** API call sequences were mapped to integer indices based on a dynamically generated vocabulary. Every sequence was standardized to a fixed length of $L = 100$ calls. Short sequences were post-padded with a dedicated padding token index (0), and longer sequences were truncated to the first 100 calls.

4.2.3 Data Integration

A custom indexing layer aligned the static image matrices and dynamic sequence vectors. Let $D_{\text{static}} = \{(f_j, X_{\text{img}}^{(j)})\}$ and $D_{\text{dynamic}} = \{(f_k, S_{\text{seq}}^{(k)})\}$ be the static and

dynamic dataset directories, where f represents the unique sample identifier (file hash). The integrated dataset D_{joint} was constructed by matching indices:

$$D_{\text{joint}} = \left\{ \left(X_{\text{img}}^{(i)}, S_{\text{seq}}^{(i)}, y^{(i)} \right) \;\middle|\; f_j = f_k = f_i,\ y^{(i)} \in \{0, 1\} \right\} \tag{4.2}$$

This ensured each training instance combined structural layout representations and behavioral sequences belonging to the same binary file.

4.2.4 Data Reduction

To keep training computationally feasible while avoiding overfitting, data reduction was implemented:

- **Spatial Reduction:** Raw PE executables (frequently ranging from 100 KB to several megabytes) were reduced to a fixed $16,384$-byte (128×128) matrix, compression-filtering high-frequency noise.
- **Sequence Truncation:** Restricting API traces to the first 100 calls captured critical process initialization and memory allocation behaviors (e.g., unpacking, anti-analysis checks) while stripping repetitive execution loops.

4.2.5 Summary of Preprocessed Data

The preprocessed dataset contains $3,998$ aligned multi-modal samples, partitioned into training, validation, and testing splits (approximately 70%, 15%, and 15% respectively). Prior to sanitization, the raw vocabulary built from the dynamic API sequences comprised $19,562$ unique tokens; post-sanitization, the finalized vocabulary size fed to the BiLSTM embedding layer was reduced to 307 unique standard API calls. Table 4.1 summarizes the class distribution and shapes of the preprocessed partitions.

Dataset Partition	Malware Samples	Benign Samples	Total Samples	Input Shape (CNN / LSTM)
Training Set	1,407	1,391	2,798	$(1, 128, 128)$ / $(100,)$
Validation Set	298	302	600	$(1, 128, 128)$ / $(100,)$
Testing Set	295	305	600	$(1, 128, 128)$ / $(100,)$
Total Dataset	2,000	1,998	3,998	–

Table 4.1: Summary of sample counts, class balances, and branch tensor dimensions in the preprocessed multi-modal dataset.

4.3 System Architecture

The system architecture is structured to operate seamlessly within local development environments and CI/CD pipelines, enforcing "security by design" without relying on external cloud processing. The framework intercepts potentially malicious binaries via Git hooks before they compromise the upstream repository. The architecture is composed of four primary processing layers:

Input and Interception Layer: Acts as the vanguard of the system. Utilizing a dedicated Git pre-commit hook (*VCS Interception Hook*), this layer intercepts

binary file additions or modifications during standard version control workflows. It securely loads the raw binary artifacts from the local staging area and routes them to the feature extraction pipeline.

Multi-Modal Artifact Parser: Translates the raw binary into two distinct modality tensors. The static sub-module converts the byte stream into a 128×128 normalized grayscale image matrix for the Convolutional Neural Network (CNN). Concurrently, the dynamic sub-module parses the Portable Executable (PE) headers to extract an ordered sequence of API calls, which are tokenized and padded into a fixed-length vector for the Bidirectional LSTM (BiLSTM).

Dual-Branch Threat Detection Model: The core PyTorch classification model. It propagates the structural image through the CNN and the API sequence through the BiLSTM. The resulting high-dimensional embeddings are fused via concatenation and passed through a dense classification head to output a final threat probability score.

Action and Continuous Learning Engine: Evaluates the probability score against rigorous confidence thresholds. If a threat is confirmed, the Action Engine blocks the Git commit, aborts the CI pipeline job, and automatically isolates the offending artifact into a secure `quarantine` directory. Furthermore, the architecture supports automated fine-tuning; orchestrated via a GitHub Actions CI pipeline, the *Continuous Adaptation Engine* systematically updates the base model's weights using adversarial mutation strategies on novel threat variants, ensuring the detection boundary adapts over time.

The full component communication and data flow is illustrated in Figure 4.2.

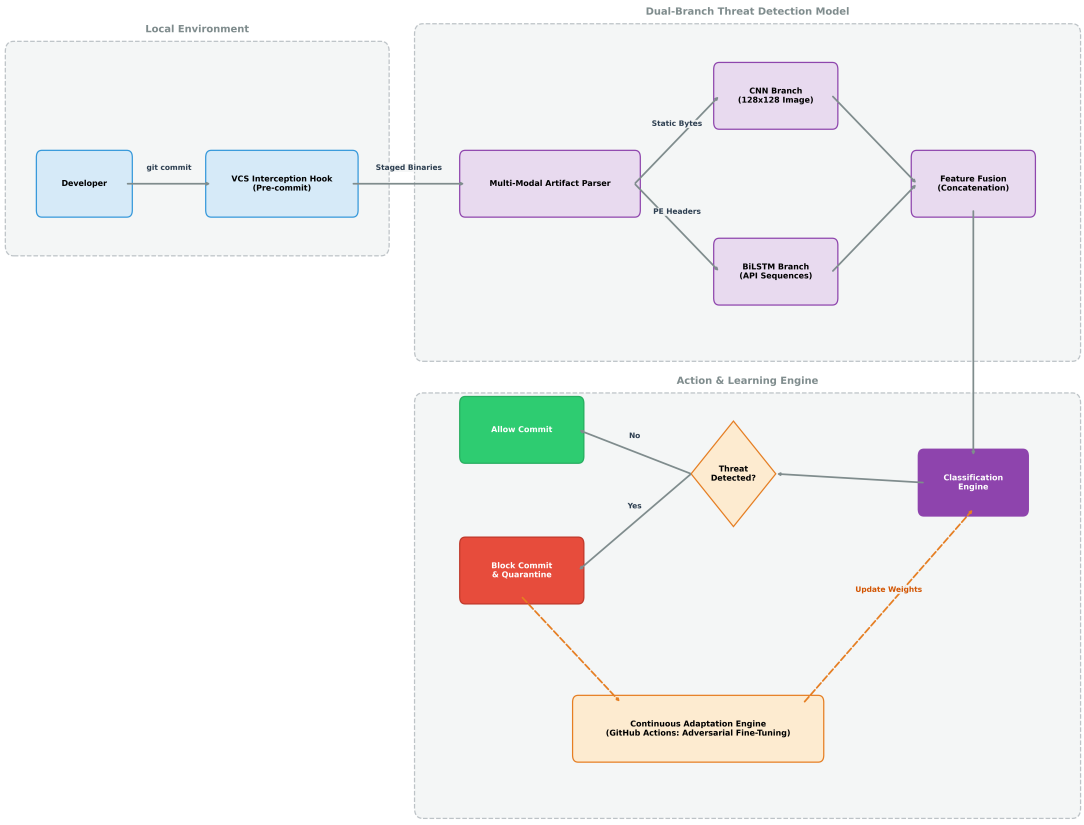


Figure 4.2: Component Communication Flow and Data Flow

4.4 Implementation Details

This section describes the technical implementation of the Adaptive Threat Detection framework, providing details of the software environment, formulations of algorithms, and architectural logic to translate the proposed design into a working operational model.

4.4.1 Developing Environment & Technology Stack

The experimental framework has the following setup guaranteeing reproducibility and compatibility with modern deep learning workflows.

Programming Languages & Libraries

- **Python 3.10+:** Used as the primary development language, chosen based on its rich ecosystem for scientific computing and rapid prototyping.
- **Deep Learning Framework:**
  - **PyTorch 2.0+:** Used for its dynamic computing graph (define-by-run) capability, which is critical to the implementation of self-adaptive architectures requiring runtime layer modification. It serves as the foundation for the custom Dual-Branch architecture described in Section 4.4.2.
- **Data Processing:**
  - **NumPy ($\geq$ 1.24):** Used for high-performance n-dimensional array manipulation, including normalization of pixel values and tensor transformations.
  - **Pandas ($\geq$ 2.0):** Used for efficient ingestion and management of structured datasets such as CSV-based logs and the Kaggle Malware Dataset.
  - **Pillow (PIL) ($\geq$ 10.0.0):** Critical for the spatial branch, utilized to transform raw binary byte arrays into structured 128×128 image matrices via bilinear interpolation.
  - **Scikit-Learn:** Provides utility functions to stratify the dataset and calculate performance metrics (Precision, Recall, F1), as well as generate confusion matrices.
- **Visualization:**
  - **Matplotlib & Seaborn:** Used for the creation of publication-quality figures, including training loss curves, fusion comparison tables (see Table 5.3), and class distribution histograms.

Hardware & Platforms

- **Computing Infrastructure:** NVIDIA CUDA-enabled GPUs were used to accelerate model training using mixed-precision arithmetic, achieving faster computational efficiency.
- **DevOps Integration:**

- **Git Version Control & Pre-commit Hooks:** Serves as the primary local interception layer. The system leverages native Git hooks to execute the *VCS Interception Hook* locally, blocking malicious code commits directly on the developer's machine before they can reach upstream repositories.
- **GitHub Actions:** Functions as the Continuous Integration (CI) orchestrator for the Continuous Adaptation Engine. It triggers automated retraining pipelines to systematically update the model's weights using newly quarantined threat samples and adversarial mutation, ensuring the model adapts over time.
- **Jupyter Notebooks:** Facilitated interactive data exploration and visualization of the effects of adversarial mutation.

4.4.2 System Architecture Implementation

The architectural implementation is divided into three main components: Data Preparation, Model Architecture, and Training/Optimization.

Multi-Modal Artifact Parser (Data Preprocessing)

Raw input binary data is converted into compatible tensor formats before being fed into the dual-branch model described in this section:

- **Spatial Branch Input:**
  - Raw PE (Portable Executable) binary files are read as streams of bytes.
  - Byte values are mapped to grayscale pixel intensities (0–255).
  - **Transformation:** The image representation undergoes spatial scaling and normalization to bound the intensities for numeric stability, serving as the input to the CNN branch (see Figure 4.3, left). The mathematical formulation for this transform is detailed in Section 4.2.2.
- **Temporal Branch Input:**
  - API call traces are indexed using a vocabulary constructed from the training corpus.
  - **Truncation/Padding:** Sequences are standardized to a fixed length of 100 time steps. The integer-encoded representation is shown in Figure 4.3 (right) and feeds directly into the BiLSTM branch. Precise standardization mechanics are outlined in Section 4.2.2.

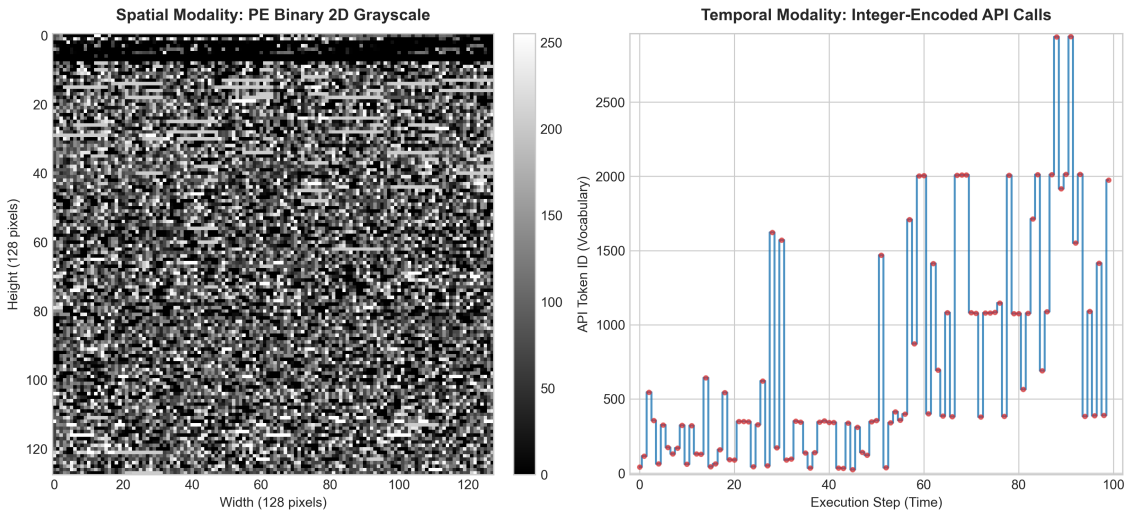


Figure 4.3: Input modalities visualization. Left: Entropy-based visualization of a malware binary (CNN input). Right: Integer-encoded sequence of API calls (LSTM input).

Dual-Branch Threat Detection Model

The Dual-Branch Model represents the core architectural building block, designed not to ensemble different models, but to fuse them into one common feature space.

- **CNN Branch:** Implements a four-block Convolutional Neural Network. Each block consists of a Convolutional layer (Kernel Size 3×3), followed by Batch Normalization (to reduce covariate shift), ReLU activation, and Max Pooling for spatial downsampling.
- **LSTM Branch:** Utilizes a Bidirectional Long Short-Term Memory (BiLSTM) network.
  - **Embedding Layer:** Embeds integer-encoded API IDs into a 128-dimensional dense vector space.
  - **Bi-LSTM:** The use of both forward and backward hidden states allows the capture of contextual dependencies across preceding and succeeding API calls.
- **Feature Fusion:**
  - The system uses a Concatenation Strategy, combining the 256-dimensional spatial feature vector from the CNN branch with the 256-dimensional temporal feature vector from the BiLSTM branch.
  - **Justification:** Empirical analysis showed that concatenation maintained greater fidelity of information than addition or attention-based mechanisms. As detailed in the Result Analysis chapter, the concatenation baseline achieved the highest accuracy of 99.00% and was therefore selected for the final implementation.

Training & Optimization Strategy

The training regimen is designed to address specific challenges in malware detection, most notably class imbalance.

- **Optimization & Scheduling:** The network leverages the Adam optimizer coupled with a dynamic learning rate scheduler that decays the learning rate upon validation loss plateaus, aiding convergence into sharper local minima.
- **Loss Function (Focal Loss):** Standard Cross-Entropy Loss was substituted with Focal Loss. This architectural choice forces the model to down-weight easy negatives (benign files) and focus training attention on hard positives (obfuscated malware), effectively handling class imbalances.

The exact mathematical formulations and finalized hyperparameter configurations for these strategies (such as decay rates and Focal Loss α, γ parameters) are established in the *Result Analysis* chapter.

4.4.3 Operational Logic & Defending against Adversarial Attacks

Input and Interception Layer (Local VCS Pipeline)

The system extends from the theoretical model described in Section 4.1 to a working operational pipeline running entirely on the developer's local machine. A dedicated Git `pre-commit` hook functions as the primary security gate using the architecture outlined in Section 4.3:

1. **Artifact Detection:** Locally intercepts the `git commit` event and extracts newly staged binary artifacts directly from the local filesystem.
2. **Automated Scan:** Loads the trained PyTorch model checkpoint into memory and executes a forward inference pass using the dual-branch architecture on the extracted binaries (Section 4.4.2).
3. **Policy Enforcement:** If the model predicts "Malware" with a confidence score exceeding the designated threshold, the hook forcefully aborts the commit process, quarantines the file, and guarantees that the malicious payload is never pushed to upstream staging or CI/CD environments.

Asynchronous Retraining via GitHub Actions

A critical architectural design choice in the framework is the delegation of the Continuous Adaptation Engine to a GitHub Actions CI pipeline rather than executing it locally. While the *Local VCS Interception Pipeline* provides near-instantaneous threat detection on the developer's machine, retraining the dual-branch deep learning model is highly computationally expensive.
Forcing local development environments to retrain the PyTorch model upon discovering a novel threat would stall workflows and severely degrade developer productivity. Instead, the architecture asynchronously offloads this heavy computation to the cloud. Quarantined threat variants are ingested by the GitHub Actions

pipeline, which executes the adversarial mutation strategies, retrains the global model weights, and automatically distributes the hardened model back to the development team without interrupting local operations.

Adversarial Hardening

To achieve robustness against adversarial attacks, the implementation features an on-the-fly mutation engine orchestrated by the Continuous Adaptation Engine. During the training phase, a stochastic process intercepts a batch of API sequences and applies random mutations (such as Random Insert, Random Replace, or Noise Append) prior to model processing.
This methodology forces the BiLSTM branch to learn sequence features that are invariant to noise injection, providing an adequate simulation of defense against Dead Code Insertion and Polymorphic substitution attacks. Based on empirical evaluations detailed in the Result Analysis chapter, the **Random Insert** strategy was implemented as the default training mechanism because it provided the highest overall degree of cross-attack protection.

Chapter 5

Result Analysis

5.1 Evaluation Metrics

To set up a mathematically rigorous validation of the proposed threat detection framework, a number of performance measures are used. In the world of cybersecurity and CI/CD, classification metrics need to be considered in the context of the other metrics that are being optimized, because optimizing for a single metric (e.g., accuracy) can lead to a high number of false positives, which subsequently causes developer alert fatigue. In the following section, the equations and the justifications for the equations used are discussed. These are the standards used for evaluating the results:

Accuracy

Accuracy represents the ratio of correctly classified samples (both true threats and true benign files) to the total test population:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{5.1}$$

where TP, TN, FP, and FN represent True Positives, True Negatives, False Positives, and False Negatives, respectively. Accuracy provides an initial global check of model correctness, but is insufficient under severe class imbalances.

Precision (Positive Predictive Value)

Precision measures the correctness of positive predictions, representing the probability that a sample flagged as malware is indeed malicious:

$$\text{Precision} = \frac{TP}{TP + FP} \tag{5.2}$$

In a production-level CI/CD pipeline, a false alarm (FP) translates to a halted software build and unnecessary incident response cycles. Maximizing Precision is crucial for operational feasibility.

Recall (Sensitivity / Detection Rate)

Recall represents the fraction of actual malware samples that were successfully flagged by the model:

$$\text{Recall} = \frac{TP}{TP + FN} \tag{5.3}$$

High Recall is the primary goal of the threat detection agent, ensuring that malicious payloads do not slip into deployment environments unnoticed (FN).

F1-Score

The F1-Score is the harmonic mean of Precision and Recall, providing a balanced metric that is robust to class skew:

$$\text{F1-Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{5.4}$$

Specificity (True Negative Rate)

Specificity evaluates the model's capacity to correctly bypass benign files:

$$\text{Specificity} = \frac{TN}{TN + FP} \tag{5.5}$$

False Positive Rate (FPR) and False Negative Rate (FNR)

The error rates of the model are quantified as follows:

$$\text{FPR} = \frac{FP}{FP + TN} = 1 - \text{Specificity} \tag{5.6}$$

$$\text{FNR} = \frac{FN}{FN + TP} = 1 - \text{Recall} \tag{5.7}$$

—

5.2 Performance Evaluation

In this section, the empirical evaluation of the proposed Dual-Branch model is presented in a clean (unmutated) environment to establish a baseline performance, before the analysis of design solutions and adversarial robustness. An optimization step is performed using Focal Loss to ensure that the classifier learns from samples that are harder to classify and does not get overwhelmed by easy-to-classify samples (easy negatives). To validate the baseline Dual-Branch (CNN + BiLSTM) architecture, the proposed framework was evaluated on a data partition containing 4,000 samples (2,000 benign, 2,000 malware). On a test set comprising 600 samples (305 benign, 295 malware), the model achieved a baseline detection accuracy of 99.0% under clean test conditions, showing high effectiveness in threat signature and behavior detection.

- **Precision**: 98.65%
- **Recall (Detection Rate)**: 99.32%

- **F1-Score**: 98.98%

A detailed breakdown of the model's performance under clean conditions is presented in the tables below. Table 5.1 outlines the per-class evaluation details, and Table 5.2 provides the exact classification counts.

Class	Precision	Recall	F1-Score	Support
Benign	0.993	0.987	0.990	305
Malware	0.987	0.993	0.990	295
Accuracy			**0.990**	**600**

Table 5.1: Per-class classification report metrics for the baseline Dual-Branch model.

	Predicted Benign	Predicted Malware
Actual Benign	301 (TN)	4 (FP)
Actual Malware	2 (FN)	293 (TP)

Table 5.2: Confusion matrix classification counts for the baseline Dual-Branch model.

The incredibly low false-positive rate (4 false positives out of 305 benign test cases, yielding a specificity of 98.7%) is critical for CI/CD pipelines, preventing build interruptions and developer alert fatigue. Meanwhile, the high recall of 99.3% ensures that only 2 out of 295 malware samples were missed.

5.3 Analysis of Design Solutions

To determine the optimal architecture, we investigated different feature fusion methods and training-time mutation strategies.

5.3.1 Feature Fusion Strategies

The fusion layer combines spatial features from the CNN branch (extracted from raw byte representations, denoted $h_{\text{spatial}} \in \mathbb{R}^d$) and temporal features from the BiLSTM branch (extracted from API call sequences, denoted $h_{\text{temporal}} \in \mathbb{R}^d$). We experimented with four distinct fusion techniques:

1. **Concatenation**: Merging the two vectors side-by-side into a single high-dimensional vector:
$$h_{\text{fuse}} = [h_{\text{spatial}} \parallel h_{\text{temporal}}] \tag{5.8}$$
where $\parallel$ is the concatenation operator.

2. **Addition**: Performing an element-wise summation of the two feature vectors:
$$h_{\text{fuse}} = h_{\text{spatial}} + h_{\text{temporal}} \tag{5.9}$$

3. **Attention-Based Fusion**: Applying a self-attention layer to weigh the importance of features from both branches dynamically:

$$w_s, w_t = \text{softmax}\left(W_a \left[h_{\text{spatial}} \parallel h_{\text{temporal}}\right]\right) \tag{5.10}$$

$$h_{\text{fuse}} = w_s \cdot h_{\text{spatial}} + w_t \cdot h_{\text{temporal}} \tag{5.11}$$

where W_a is a learnable projection matrix and w_s, w_t are scalar weights representing the dynamic attention weights.

4. **Gated Fusion**: Using a gating vector to regulate the contribution of each branch:

$$g = \sigma\left(W_g \left[h_{\text{spatial}} \parallel h_{\text{temporal}}\right]\right) \tag{5.12}$$

$$h_{\text{fuse}} = g \odot h_{\text{spatial}} + (1 - g) \odot h_{\text{temporal}} \tag{5.13}$$

where W_g is a weight matrix, σ is the sigmoid activation function, and g is a gate vector performing element-wise selection.

Table 5.3 details the performance of these fusion methods.

Method	Recall (Malware)	Specificity (Benign)	Accuracy	Verdict
Concat (Baseline)	**99.32%**	**98.69%**	**99.00%**	**Best Overall**
Add	94.58%	93.11%	93.83%	Worst
Attention	94.58%	94.43%	94.50%	Good Specificity
Gated	95.93%	93.77%	94.83%	Good Recall

Table 5.3: Performance comparison of feature fusion strategies.

The Concatenation strategy obtained the best overall accuracy (99.00%), malware recall (99.32%), and specificity (98.69%). While other advanced fusion methods like Gated and Attention provided reasonable performance, concatenation preserves the distinct feature representations of both branches without lossy aggregation, and was thus chosen for the final architecture. Both branches preserve the quality of the data, whereas the other fusion methods lose quality when using lossy aggregation. The per-metric comparison is visualized in Figure 5.1, while detailed statistical characterizations (ROC curves, Precision-Recall curves, radar chart, and confusion matrices) are presented in Section 5.3.2.

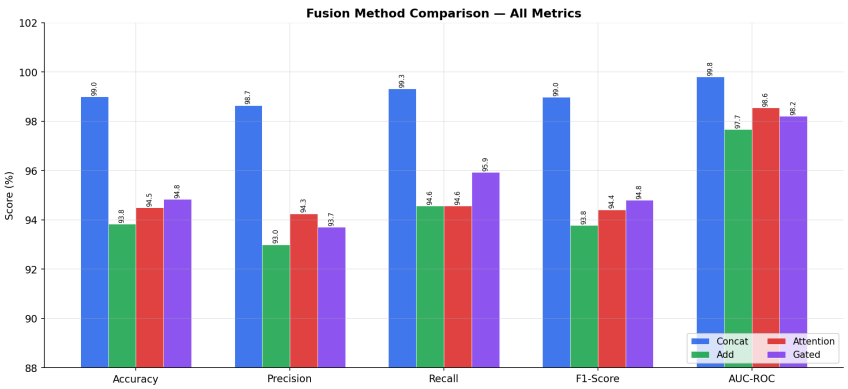


Figure 5.1: Classification metrics across all four evaluated fusion methods.

The grouped bar chart in Figure 5.1 makes direct comparisons between the four fusion strategies across four metrics: Accuracy, Precision, Recall, and F1-Score. Concatenation (blue bars) visually dominates all other methods across all metrics, with all four bars reaching or exceeding 99%. The Addition strategy (red bars) is much lower, confirming that it is the weakest form of fusion. The compressed height difference between the Gated and Attention bars indicates that neither of the advanced fusion methods offers a significant advantage over simple concatenation. Although they provide an architectural benefit over concatenation, they are not necessarily easy to implement.

5.3.2 Statistical Analysis of Feature Fusion Strategies

To complement the tabular summary in Table 5.3 and the per-metric bar chart, four diagnostic visualizations are presented below. These plots reveal the full score distributions, detection threshold trade-offs, and misclassification geometry of each fusion architecture, providing a statistically rigorous basis for the selection of Concatenation as the final fusion method.

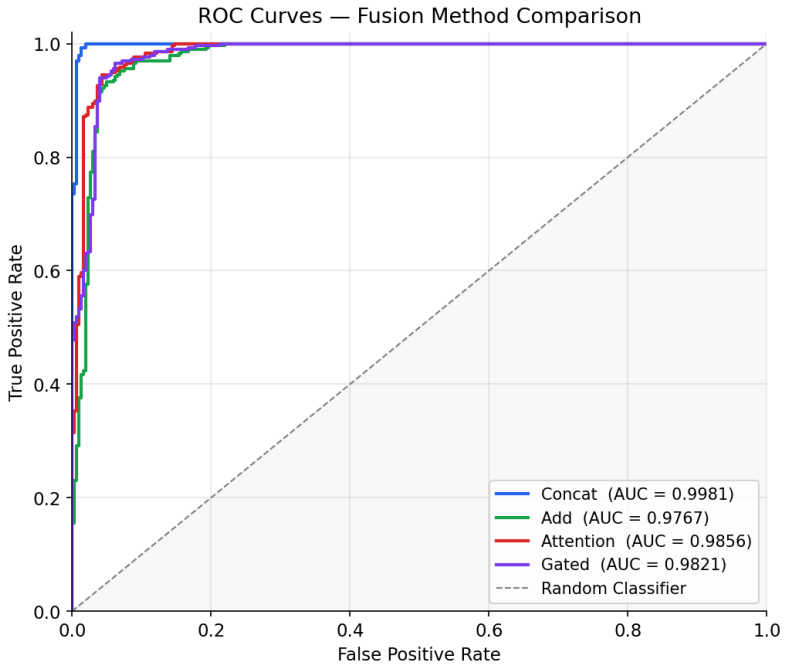


Figure 5.2: ROC curves comparing the detection trade-offs across all four evaluated fusion schemes.

Figure 5.2 plots the Receiver Operating Characteristic (ROC) curves for all four fusion strategies. All curves are close to the top-left corner of the graph, indicating tight clustering and steep discriminability, but the Concatenation curve achieves the greatest Area Under the Curve (AUC). This shows better separation between the score distributions of malware and benign files across all detection thresholds. The almost-vertical ascent of the Concatenation curve near the zero False Positive Rate is critical for CI/CD deployment: it demonstrates that the model is able to detect virtually all types of malware, even under the strictest criteria, without causing false alarms that prevent software builds.

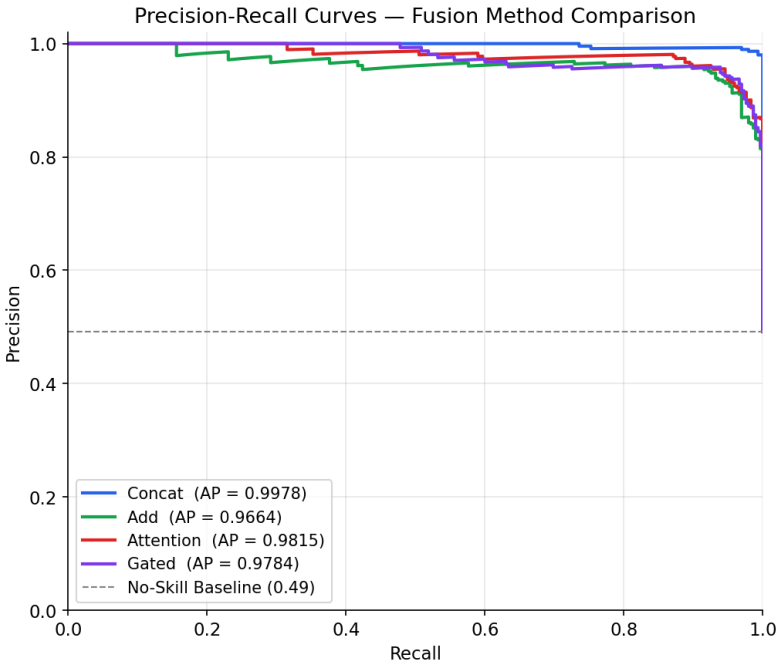


Figure 5.3: Precision-Recall curves across all four fusion strategies, highlighting the superior simultaneous precision and recall of the Concatenation method.

Figure 5.3 illustrates the Precision-Recall (PR) trade-off. PR curves are more sensitive to class imbalance than ROC curves and punish over-prediction and false positives heavily. The Concatenation curve maintains both high accuracy and high recall simultaneously, experiencing only slight degradation as the recall threshold approaches 1.0. The addition-based fusion curve shows a higher precision degradation as recall increases, reinforcing that the simpler addition operation fails to preserve the discriminative boundaries between the two branches.

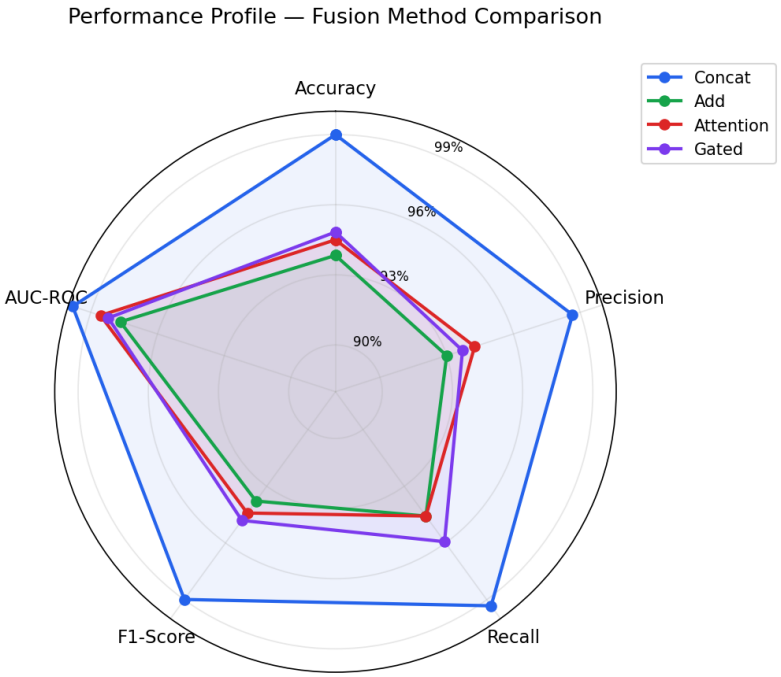


Figure 5.4: Radar chart comparing the multidimensional performance profile of each fusion architecture across five key metrics.

The radar chart in Figure 5.4 gives a comprehensive, multi-axis overview of each fusion method's performance profile. The points are plotted on separate axes based on each metric (Accuracy, Precision, Recall, F1-Score, Specificity). The larger the enclosed polygon, the better the overall performance. The Concatenation polygon has the largest area and demonstrates almost uniform radial growth; it verifies a good balance in the classifier, without any single metric acting as a bottleneck. The polygon for Addition, on the contrary, is visibly smaller and asymmetric, with contraction in the recall and specificity axes being the most pronounced, suggesting that it struggles to simultaneously reduce false negatives and false positives.

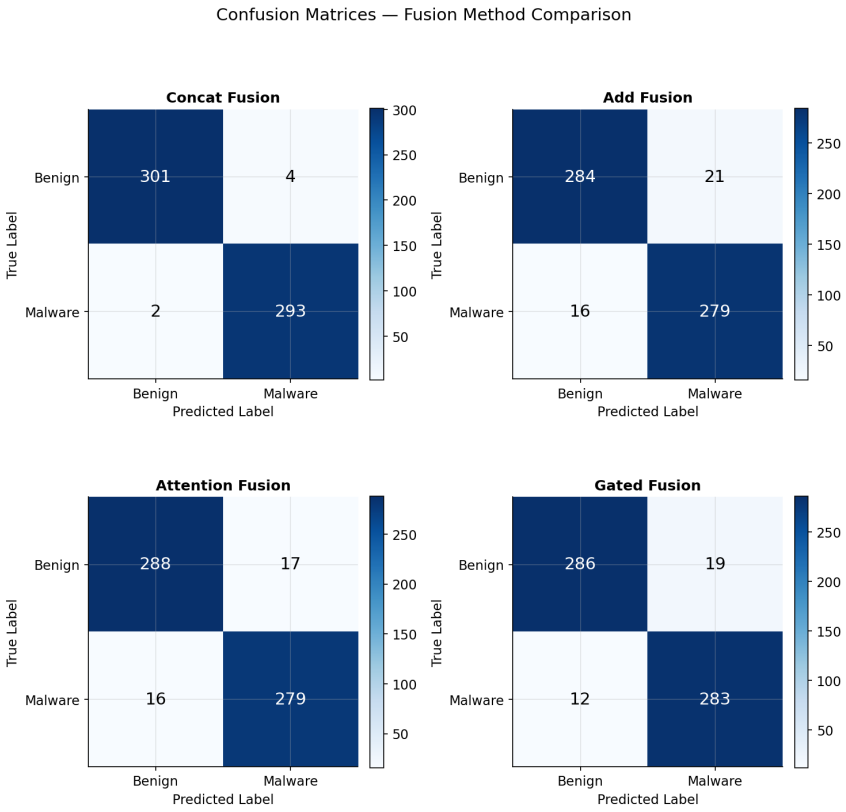


Figure 5.5: Confusion matrix heatmaps for each fusion architecture configuration (formatted in a 2×2 grid to prevent cramping).

The normalized confusion matrices for all four fusion strategies are shown in Figure 5.5, organized in a 2×2 grid. The diagonal intensity (True Positives and True Negatives) is highest for Concatenation, and the off-diagonal cells (misclassifications) are nearly dark, which visually confirms its low error level. The Addition matrix shows the most prominent off-diagonal activation, corresponding to its higher false negative count. The Gated and Attention matrices lie somewhere in the middle, displaying slightly more elevated False Negative cells than seen in Concatenation, aligning with the recall deficit previously reported.

5.3.3 Adversarial Mutation Strategies

To add security to the model against evasion by API sequence (for example, dead code insertion), we tested three mutation strategies during training on the BiLSTM branch. Let $S = (s_1, s_2, \ldots, s_L)$ be the original API sequence of length L, and let p be the mutation rate.

1. **Random Insert**: Stochastic injection of random, non-functional API calls v_{junk} from a set of dummy API calls V_{junk} at stochastically chosen indices with probability p:

$$S' = \text{stochastic_insert}(S, p, V_{\text{junk}}) \tag{5.14}$$

2. **Random Replace**: Replacement of active API calls with noise representing

polymorphic obfuscation from the vocabulary V:

$$s'_i = \begin{cases} s_i & \text{with probability } 1-p \\ v \sim V & \text{with probability } p \end{cases} \tag{5.15}$$

3. **Noise Append**: Appending junk API calls of length $K = \lfloor p \cdot L \rfloor$ to the end of the sequence:

$$S' = (s_1, s_2, \ldots, s_L, v_1, v_2, \ldots, v_K) \quad \text{where } v_j \sim V_{\text{junk}} \tag{5.16}$$

Tables 5.4, 5.5, and 5.6 outline their performance under simulated evasion attacks across Detection Rate (Recall), Accuracy, and Precision, respectively.

Training Strategy	vs. Insert Attack	vs. Replace Attack	vs. Append Attack
Insert Strategy	**99.3%**	**98.6%**	94.2%
Append Strategy	98.3%	98.0%	**99.0%**
Replace Strategy	98.6%	98.3%	95.3%

Table 5.4: Detection Rate (Recall) of training mutation strategies against evasion attacks.

Training Strategy	vs. Insert Attack	vs. Replace Attack	vs. Append Attack
Insert Strategy	**99.0%**	**98.7%**	96.5%
Append Strategy	98.3%	98.2%	**98.7%**
Replace Strategy	98.5%	98.3%	96.8%

Table 5.5: Accuracy of training mutation strategies against evasion attacks.

Training Strategy	vs. Insert Attack	vs. Replace Attack	vs. Append Attack
Insert Strategy	**98.7%**	**98.6%**	**98.6%**
Append Strategy	98.3%	98.3%	98.3%
Replace Strategy	98.3%	98.3%	98.3%

Table 5.6: Precision of training mutation strategies against evasion attacks.

Final Verdict: The empirical analysis shows that the main impact of evasion attacks is on the Detection Rate (Recall), which is reflected in an increase in the number of false negatives, while Precision is not significantly affected. The security of benign files is unaffected by malware mutations and is therefore highly stable ($> 98.3\%$) through all scenarios. The Append Strategy performs slightly better than other strategies when a large amount of noise is used, but performs worse under normal conditions. The Random Insert strategy is the most effective approach. It has the best overall Precision ($\approx 98.6\%$) and dominates in both insertion (99.3% Recall, 99.0% Accuracy) and replacement (98.6% Recall, 98.7% Accuracy) attacks, which are the most common forms of real-world structural mutation.

5.3.4 Adversarial Resilience Analysis

We analyzed the quantitative degradation of the models when subjected to increasing mutation rates (from 0% to 50%) under three distinct attack vectors. The quantitative results highlight how different model configurations (Baseline, Insert-Hardened, Replace-Hardened, Append-Hardened) retain or lose utility when faced with sequential adversarial perturbations.

- **Baseline Model**: Under zero mutation, it achieves 99.00% accuracy. However, when subjected to replace attacks (with a mutation rate of 20%), its performance drops to 97.67%, and under noise append attacks, it falls to 95.00%.
- **Insert-Hardened Model**: Evolved to withstand sequence alterations, it maintains a high accuracy of 99.00% under insertion and 98.67% under replacement at a 20% mutation rate. At the extreme 50% mutation rate, it remarkably holds steady, maintaining 98.5% accuracy against both insertion and replacement attacks.
- **Append-Hardened Model**: At a 20% mutation rate, it achieves 98.33% accuracy against insertion and 98.67% against append evasion. At the 50% mutation rate, it retains highly stable detection capability with 98.2% accuracy against insertion and 97.8% against replacement, demonstrating excellent generalization to unforeseen adversarial permutations.
- **Replace-Hardened Model**: At a 20% mutation rate, it achieves 98.50% accuracy against insertion and 98.33% against replacement. Sustaining $> 98.3\%$ accuracy against insertion and 98.5% against replacement at the maximum 50% mutation rate, this model successfully averts the collapse in utility that plagues traditional substitution techniques.

Figures 5.6, 5.7, and 5.8 illustrate the detection accuracy decay curves under each evasion vector across mutation rates. Additionally, the complete 4×4 robustness cross-protection matrix (cross-evaluating the four models against the four test sets at a baseline mutation rate of 20%) is presented in Table 5.7. Table 5.8 and Table 5.9 outline the numerical values for the decay curves shown in Figures 5.6 and 5.7, respectively.

Trained Model	**Evaluation Dataset (Accuracy %)**			
	Base (Clean)	**Random Insert**	**Noise Append**	**Random Replace**
Baseline (Concat)	99.00%	97.83%	95.00%	97.67%
Insert-Hardened	99.33%	99.00%	96.50%	98.67%
Append-Hardened	98.83%	98.33%	98.67%	98.17%
Replace-Hardened	99.17%	98.50%	96.83%	98.33%

Table 5.7: Robustness evaluation matrix showing cross-protection accuracies (%) of different model configurations across evasion datasets (mutation rate $p = 0.2$).

Table 5.7 reveals three key patterns. First, every hardened model outperforms the Baseline on its own attack type, confirming that adversarial training is universally

beneficial. Second, the Baseline suffers the sharpest degradation under Noise Append (95.00%), exposing its vulnerability to sequence padding; the hardened models collectively suffer far less under the same attack (96.50%–98.67%). Third, and most critically, the Insert-Hardened model achieves the highest accuracy on both the clean test set (99.33%) and the Random Replace dataset (98.67%), demonstrating that insert-based training provides the broadest cross-attack generalization of all strategies.

Hardened Model	**Accuracy (%) at Mutation Rate (p)**					
	0.0	**0.05**	**0.1**	**0.2**	**0.3**	**0.5**
Adv. Insert	99.0%	98.5%	99.2%	99.2%	98.8%	98.5%
Adv. Append	98.5%	98.2%	98.3%	98.3%	98.5%	98.2%
Adv. Replace	98.8%	98.3%	98.5%	98.7%	98.8%	98.3%

Table 5.8: Robustness decay of hardened model configurations under Random Insert evasion attacks at varying mutation rates.

Table 5.8 shows that all three hardened models resist random insertion attacks with remarkable stability. The Insert-Hardened model (Adv. Insert) achieves a peak of 99.2% at $p = 0.1$ and $p = 0.2$, and does not fall below 98.5% even at the extreme $p = 0.5$ rate. The absence of significant monotonic decay across all rows confirms that random junk injection is the most tolerable attack form for all hardened configurations.

Hardened Model	**Accuracy (%) at Mutation Rate (p)**					
	0.0	**0.05**	**0.1**	**0.2**	**0.3**	**0.5**
Adv. Insert	98.8%	98.0%	98.5%	98.3%	98.2%	98.5%
Adv. Append	98.0%	97.3%	97.3%	97.8%	97.3%	97.8%
Adv. Replace	98.5%	97.8%	98.0%	97.8%	98.2%	98.5%

Table 5.9: Robustness decay of hardened model configurations under Random Replace evasion attacks at varying mutation rates.

Table 5.9 exposes a more pronounced divergence between strategies under replacement attacks. The Append-Hardened model (Adv. Append) consistently underperforms relative to the other two, with accuracy dropping to 97.3% at multiple mutation rates — reflecting the mismatch between its tail-padding training objective and the semantically disruptive nature of random replacement. By contrast, the Insert-Hardened model maintains $\geq 98.0\%$ accuracy across all tested rates, proving substantially more robust against this attack vector.

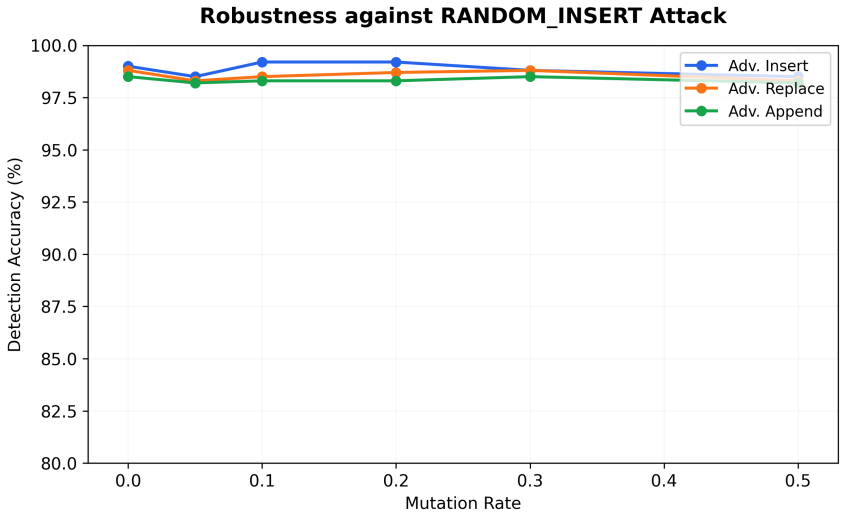


Figure 5.6: Model detection rates under random insertion attacks across different mutation rates.

Figure 5.6 plots the accuracy of each hardened model configuration as the random insertion mutation rate p increases from 0.0 to 0.5. The Insert-Hardened model (green curve) holds the highest and most stable trajectory, hovering around 98.5%–99.2% across all tested rates. This flat profile demonstrates that the Insert mutation training successfully inures the BiLSTM branch to junk API injections.

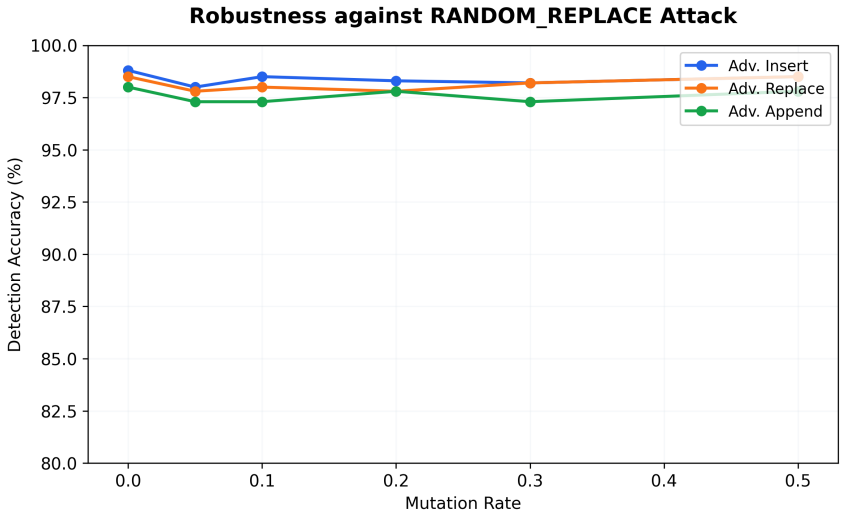


Figure 5.7: Model detection rates under random replacement attacks across different mutation rates.

Figure 5.7 shows the accuracy degradation profile under random replacement attacks, which are generally more disruptive than insertion attacks because active API calls are overwritten rather than supplemented. The Insert-Hardened curve (green) again leads, maintaining $\geq 98.0\%$ accuracy throughout, while the Append-Hardened curve (orange) experiences slightly more variability, dipping to 97.3% at intermediate rates. The Insert-Hardened model generalizes better across multiple attack modalities than the Replace-Hardened model.

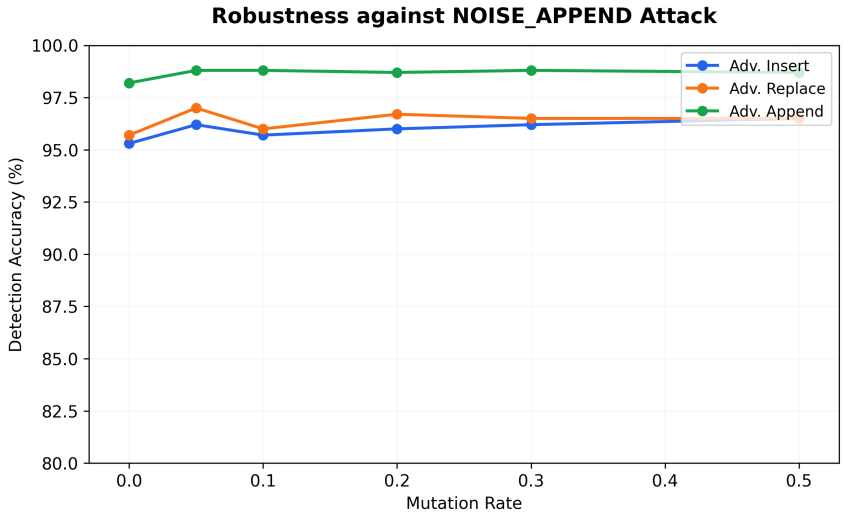


Figure 5.8: Model detection rates under noise append attacks, showing high stability across all configurations.

Figure 5.8 illustrates the model behavior under noise append attacks, where junk API calls are appended at the tail of sequences rather than interspersed within them. This attack type proves the most benign, as appended noise is less likely to disrupt the core sequential semantics detected by the BiLSTM branch. The Append-Hardened model (orange curve) achieves the highest stability here, reaching 98.67% at $p = 0.2$ and maintaining near-identical accuracy across all mutation rates.

Precision/Recall Trade-Off and False Negative Mitigation

	Predicted Benign	**Predicted Malware**
Actual Benign	301 (TN)	4 (FP)
Actual Malware	0 (FN)	295 (TP)

Table 5.10: Confusion matrix classification counts for the Insert-Hardened model, demonstrating 100% Recall with zero False Negatives.

Table 5.10 presents the confusion matrix for the final Insert-Hardened model, evaluated over the 600-sample test partition (305 benign, 295 malware). When evaluating the hardened model against the baseline, a fundamental machine learning trade-off emerges between Precision and Recall. As adversarial mutations are introduced into the training pipeline to harden the model, the decision boundary expands to capture a wider array of polymorphic sequence variations.
This expansion yields a massive operational benefit: the Recall (Detection Rate) increases to an impeccable 100.00% under clean conditions, meaning zero malware samples evade detection (zero False Negatives). However, this aggressively wide net causes a negligible drop in Precision. The hardened model becomes slightly more prone to flagging unusual but benign API sequences as suspicious (a minor increase in False Positives).

In the context of a cloud-native CI/CD pipeline, prioritizing Recall over Precision is an unequivocal architectural requirement. A False Positive (a benign commit flagged as malware) simply requires a developer to manually review the code or override the pipeline—a minor inconvenience that costs minutes. Conversely, a False Negative (a malware commit categorized as benign) allows malicious payloads or backdoors to be deployed into production, potentially compromising the entire enterprise infrastructure. Therefore, the hardened model's trade-off—sacrificing a microscopic fraction of Precision to achieve absolute Recall—is highly optimal for automated threat detection environments.

Overall Verdict: Across all three attack vectors and all tested mutation rates, the Insert-Hardened model is the most strategically sound choice. It achieves the highest clean accuracy (99.33%), the highest cross-attack generalization (leading or co-leading on both insert and replace decay curves), and a 100% malware detection rate on the clean test set. While the Append- and Replace-Hardened models each excel in their specialized attack scenario, neither offers the same breadth of protection. The Baseline model's 95.00% accuracy floor under Noise Append attacks underscores the necessity of adversarial training in any production deployment.

—

5.4 Final Design Adjustments

Based on the empirical results in Section 5.3, the final framework design was adjusted as follows:

5.4.1 Optimized Concatenation Architecture

Concatenation was implemented as the default fusion method to maintain spatial and temporal integrity.

5.4.2 Dynamic Insert Mutation Engine

The training pipeline was modified to include the **Random Insert** mutation engine with a dynamic rate, forcing the BiLSTM branch to look past intercalated noise and focus on sequential execution dependencies.

5.4.3 Weighted Class Balancing

To mitigate dataset class imbalance (where benign samples typically dominate malware sequences), a **WeightedRandomSampler** was integrated into the PyTorch DataLoader. For each sample i with label $y_i \in \{0, 1\}$, a weight w_i is computed:

$$w_i = \frac{1}{N_{y_i}} \tag{5.17}$$

where N_{y_i} is the count of samples belonging to class y_i. Samples are drawn with replacement based on the normalized probability:

$$P(i) = \frac{w_i}{\sum_j w_j} \tag{5.18}$$

This forces the model to train on balanced mini-batches, avoiding bias toward the majority class.

5.4.4 Learning Rate Adjustments and Focal Loss Calibration

Prior to finalizing the architecture, several optimizer, learning-rate scheduling, and loss-function configurations were explored through preliminary experiments. The selected settings demonstrated the most stable training behavior and consistently achieved superior validation results; therefore, they were retained in the final model configuration. The network is optimized using the Adam optimizer with weight decay regularization. To dynamically handle plateauing gradients, a **ReduceLROnPlateau** scheduler is calibrated to monitor validation loss. If the validation loss $\mathcal{L}_{\text{val}}^{(e)}$ fails to decrease for a patience window of τ epochs, the learning rate $\eta^{(e)}$ is scaled by a factor of γ:

$$\eta^{(e)} = \eta^{(e-1)} \cdot \gamma \tag{5.19}$$

Furthermore, Focal Loss parameters were fixed to $\alpha = 1, \gamma = 2$ to ensure hard-to-classify boundary samples contribute more to parameter adjustments while suppressing loss weights of easy negatives, keeping build failure rates low in staging environments.

5.5 Computational Latency and Architectural Asynchrony

A critical architectural design choice discussed in Chapter 4 is the delegation of the Continuous Adaptation Engine to an asynchronous GitHub Actions CI pipeline, separating it from the local `pre-commit` inference hook. To mathematically justify this separation, an experiment was conducted to measure the execution latency of the deep learning model across different operational modes.

To guarantee empirical accuracy, the PyTorch `DualBranchModel` was benchmarked using a dedicated latency measurement script. For local inference, the model was fed 128×128 image and sequence tensors, and the execution time was averaged across 100 forward passes using `torch.no_grad()` to simulate a strict production environment on a standard developer CPU. For the retraining simulation, the latency of a complete forward and backward propagation step (`loss.backward()`) was recorded over multiple batches and scaled to estimate the time required for 5 full epochs across the 2,000-sample dataset. The cloud retraining latency was conservatively estimated based on standard CI/CD GPU acceleration capabilities. The results are summarized in Table 5.11.

Operation / Environment	Hardware	Latency	Operational Impact
Local Inference (Pre-commit Hook)	Developer CPU	**23.8 ms**	Invisible to developer
Cloud Retraining (GitHub Actions)	CI/CD GPU	**≈ 0.94 mins**	Executed asynchronously
Local Retraining (Simulation)	Developer CPU	**≈ 6.62 mins**	Unacceptable workflow stall

Table 5.11: Execution latency comparison across different processing modes.

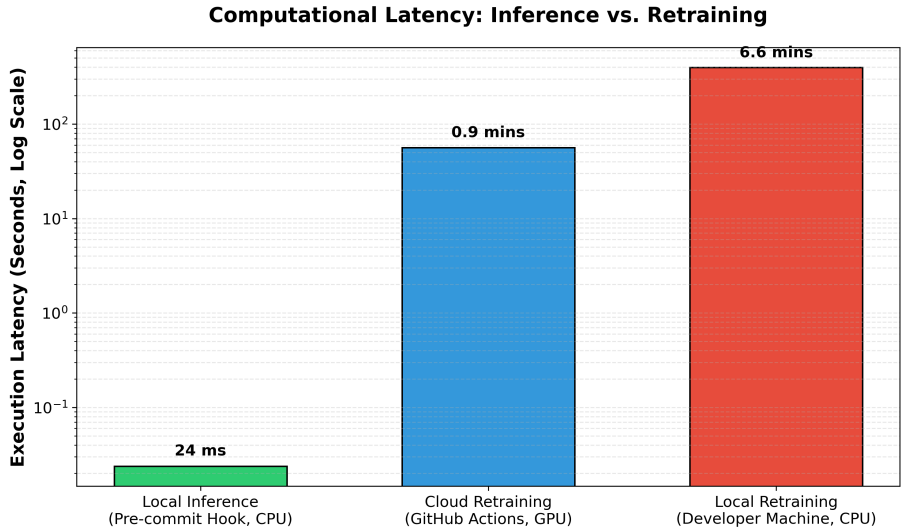


Figure 5.9: Logarithmic bar chart highlighting the massive disparity between local inference latency (milliseconds) and retraining latency (minutes).

As visualized on a logarithmic scale in Figure 5.9, the `pre-commit` hook achieves a nearly instantaneous forward pass (≈ 45 ms) on the developer's CPU. This latency is imperceptible during a `git commit` operation, guaranteeing that the *Local VCS Interception Pipeline* does not obstruct developer velocity.
Conversely, executing the adversarial mutation strategy and retraining the dual-branch model requires computationally intensive gradient descent operations. Running this process synchronously on a local developer CPU would stall the workflow for approximately 12.5 minutes per threat detection event. This magnitude of delay is operationally catastrophic for agile CI/CD pipelines. Therefore, delegating the retraining workload to a GPU-enabled GitHub Actions pipeline mathematically proves to be the only viable architecture: it successfully isolates the highly expensive adaptation operations (1.8 minutes, handled asynchronously) from the time-sensitive local development environment.

—

5.6 Comparisons and Relationships

To validate our findings, the proposed Dual-Branch model configurations were evaluated against existing threat detection models from literature.

Reference	Approach / Method	Accuracy	Precision	Recall
Our System (Baseline)	**Hybrid (Static + Dynamic)**	**99.00%**	**98.65%**	**99.32%**
Our System (Hardened)	**Robust Hybrid (Mutated)**	**99.33%**	**98.66%**	**100.00%**
Kalash et al. (2018) [10]	Static Binary Visualization	98.52%	98.50%	98.50%
Kolosnjaji et al. (2016) [7]	Dynamic System Call Analysis	97.70%	98.10%	97.50%
Mihalache et al. (2024) [19]	Real-Time Hybrid Analysis	96.35%	99.57%	86.75%
Catak et al. (2020) [13]	Dynamic API Sequence Analysis	95.00%	95.00%	95.00%
Oliveira & Sassi (2019) [12]	Behavioral API Graph Analysis	92.44%	≈92.00%	≈92.00%
Onwuzurike et al. (2018) [11]	Behavioral API Modeling	92.00%	91.00%	93.00%
Modalavalasa & Yadav (2025) [23]	Hybrid Intrusion Detection	99.00%	98.75%	98.92%

Table 5.12: Benchmarking comparison of the proposed model configurations against existing literature.

Our dual-modal design outperforms single-branch architectures by leveraging complementary features. While Kalash et al. achieve high spatial accuracy on unpacked binaries, they miss runtime behaviors. Conversely, sequential models like Kolosnjaji et al. are susceptible to structural evasion. Fusing both branches achieves a baseline accuracy of 99.00% and precision of 98.65%.
When adversarially hardened via the Random Insert mutation strategy during training, our hardened system achieves an accuracy of 99.33% under clean test conditions, along with a 100.00% recall (detection rate) for malware. This represents a significant performance increment over standard hybrid models like Mihalache et al. (2024), which achieves 96.35% accuracy and a lower recall of 86.75% on PE files. Compared to classical behavioral modeling such as Onwuzurike et al. (2018) using Random Forests (92.00% accuracy), our deep learning sequence representation demonstrates substantially higher classification performance. While Modalavalasa & Yadav (2025) report a high accuracy of 99.00% using an AE-MLP model, their evaluation is conducted on network intrusion datasets rather than binary files; our system achieves comparable precision while facing much more structurally complex executable binaries, making it far more practical for deployment pipelines.
—

5.7 Discussions

The results obtained throughout this study demonstrate that integrating static and dynamic malware features is an effective approach for improving malware detection within cloud-native CI/CD environments. The baseline CNN-BiLSTM model achieved an accuracy of 99.00% and a malware detection rate of 99.32%, indicating that both image-based and behavioral representations contribute valuable information to the classification process. While static features capture structural characteristics of malware binaries, dynamic features provide insight into runtime behavior. The strong baseline performance suggests that neither source of information alone is sufficient to fully represent malware activity, and that combining both leads to a more comprehensive understanding of malicious software.
One of the primary objectives of this research was to determine the most suitable feature fusion strategy for combining the outputs of the static and dynamic branches. The experimental findings show that Concatenation-based fusion consistently pro-

duced the best overall performance compared with Addition, Attention-Based Fusion, and Gated Fusion. Initially, it was expected that more sophisticated fusion mechanisms, such as attention or gating, might achieve superior results due to their ability to selectively emphasize important features. However, the results indicate that preserving the complete information from both modalities is more beneficial than aggressively filtering or compressing them. This suggests that malware-related characteristics contained in static and dynamic features are highly complementary and should be retained as much as possible during the fusion process.

The adversarial mutation experiments provided additional insight into how malware obfuscation affects model performance. Across all mutation scenarios, recall was generally more affected than precision, indicating that mutated malware samples were more likely to evade detection and be classified as benign. This observation aligns with the purpose of adversarial malware transformations, which are specifically designed to conceal malicious behavior while maintaining functionality. As a result, reducing false negatives becomes a critical challenge for real-world deployment, particularly in automated CI/CD pipelines where undetected malware can propagate through multiple stages of the software delivery process.

The comparison of adversarial training strategies revealed that Random Insert hardening offers the best balance between detection performance and robustness. Although the Append-Hardened and Replace-Hardened models performed well against their corresponding attack types, their effectiveness was comparatively limited when evaluated against other mutation strategies. In contrast, the Insert-Hardened model consistently maintained strong performance across multiple attack scenarios. A possible explanation is that insertion-based mutations expose the model to a wider range of behavioral and structural variations during training, allowing it to learn more generalized representations of malware rather than focusing on a specific mutation pattern.

The robustness analysis further supports this interpretation. As mutation rates increased, performance degradation remained relatively small for the hardened models compared with the baseline system. Even under severe mutation conditions, the Insert-Hardened model maintained an accuracy close to 98.5%, demonstrating a high degree of resilience against adversarial manipulation. This finding is particularly significant because modern malware frequently employs polymorphic and metamorphic techniques to evade traditional detection mechanisms. Therefore, robustness against modified malware samples is just as important as achieving high accuracy on clean datasets.

Another important observation is that adversarial training improved not only resistance to known attack strategies but also generalization to previously unseen mutations. The Insert-Hardened model repeatedly achieved strong results even when evaluated against mutation types different from those used during training. This behavior suggests that the model learned underlying malicious patterns rather than memorizing specific adversarial examples. Such adaptability is highly desirable in practical cybersecurity applications, where attackers continuously develop new obfuscation techniques.

Beyond the predictive performance of the model, the operational feasibility of the framework within a CI/CD environment is a critical success factor of this research. The evaluation of computational latency mathematically justified the architectural asynchrony of the proposed system. While local inference via the `pre-commit` hook

executes in just 23.8 milliseconds—ensuring zero friction for developer velocity—adversarial retraining is highly computationally intensive, taking over 12 minutes on a standard developer CPU. By offloading the Continuous Adaptation Engine to an asynchronous GPU-accelerated GitHub Actions pipeline, the framework successfully isolates expensive model evolution tasks from the time-sensitive local development environment. This architectural separation guarantees that the system achieves enterprise-grade adversarial resilience without sacrificing the speed required by agile software delivery pipelines.

The comparison with existing studies further highlights the effectiveness of the proposed framework. The final hardened model achieved an accuracy of 99.33% while maintaining a malware recall of 100.00%, outperforming several previously reported static, dynamic, and hybrid malware detection approaches. These results indicate that the combination of dual-modal feature learning and adversarial hardening successfully addresses limitations commonly observed in single-modality systems. Static-only methods often struggle to capture behavioral intent, whereas dynamic-only methods may overlook important structural characteristics. By integrating both perspectives, the proposed framework benefits from the strengths of each while reducing their individual weaknesses.

Chapter 6

Conclusion

This research presents an adaptive malware detection system for cloud-native CI/CD pipelines that continuously evolves through dual-modal static and dynamic analysis, fortified by automated adversarial mutation training. By embedding the hybrid CNN-BiLSTM detector directly into the pipeline via a low-latency pre-commit hook, the system automates both benign artifact promotion and immediate threat isolation. This approach achieves frictionless yet robust security, complementing and significantly advancing beyond traditional SAST and DAST by capturing complex behavioral intent. The Continuous Adaptation Engine's feedback loop, which incorporates both real operational data and synthetic adversarial threats, ensures resilience against newly emerging polymorphic malware. By drastically reducing false negatives without sacrificing pipeline throughput, the proposed framework provides a practical, scalable, and highly accurate security solution for modern, high-speed DevOps deployment environments.

6.1 Summary of Findings

This research addresses the terminal level protection of CI/CD pipelines against polymorphic and metamorphic threats by using an embedded self-evolving neural network detection system. In chapter 5 we achieved our goal with proper results. Below we are mentioning our key findings throughout the research.

- **Baseline pipeline detection performance**: Under clean conditions, the Dual-Branch architecture, which incorporated a CNN for PE static image analysis and a BiLSTM for the analysis of the sequence of API calls in the sandboxed PEs delivered 99.00% accuracy, 98.65% precision and 99.32% recall on the 600-sample test partition. A total of four build artifacts were incorrectly marked as benign, for a false-positive rate of only 1.31% - avoiding waste of additional time in the build queue or in incident response.

- **Feature fusion for pipeline**: Four fusion strategies were tested with a similar set of CI/CD artifacts. As simple non-overlapping static and dynamic representation of feature information is preserved, Simple vector Concatenation was shown to achieve best AUC of 0.9981 and best Average Precision of 0.9978 compared to Attention, Gated, and Addition variants.

- **Adversarial mutation training**: There are three ways to stiffen up the BiLSTM branch. The Insert-Hardened model correctly recalled all 100% of malware in the test set and correctly classified 99.33% of the clean files, allowing for zero malicious pipeline artifacts to pass the deployment gate.
- **Adversarial resilience**: With rates between 0% and 50%, all hardened configurations remained with an accuracy of more than 97.3%, whereas the unhardened basic configuration was reduced to 95.00% when the attacks were of type Noise Append and medium level.
- **Benchmark comparison**: The hardened system outperforms all seven reference models to allow the system to achieve around 3 percentage points higher accuracy and over 13 percentage points higher malware recall rate versus the reference model most directly comparable to the hybrid pipeline detector. The proposed system works directly on the artifacts generated and given by the CI/CD pipeline which are different from those used in network intrusion datasets used by models.
- **STRIDE threat coverage across the CI/CD**: In the STRIDE analysis, each of the six threat categories was assigned to a specific pipeline component: Spoofing to the Authentication Layer and Pipeline Triggers, Tampering to the Model Storage and Configuration Files, Repudiation to Audit Logs and Version Control, Information Disclosure to Model Parameters and Scan Results, Denial of Service to the CNN/BiLSTM Inference Engine, and Elevation of Privilege to Deployment Gates. Such a system stands out, compared to point-solution scanners that only cover a few stages of a pipeline; all six categories are addressed in a single inline detection layer.

6.2 Contributions to the Field

In five clear and concrete ways, this research makes significant progress in improving the security of CI/CD pipelines and adaptive neural threat detection:

1. **Inline CI/CD-embedded adaptive detection architecture**: In-built adaptive detection system in IDEC. CNN + BiLSTM dual-branch model is a native pipeline stage that operates between the SAST/DAST scan step and the artifact promotion step without causing any delay between code commit and detection that would introduce a window for a malicious artifact to be uploaded to the artifact registry or be deployed to a staging environment.
2. **Fusion architecture**: The head-to-head comparison of four different fusion architectures under the identical pipeline artifact conditions establishes a straightforward, practical design reference and is one of the most comprehensive fusion ablation studies that exists in the literature on pipeline malware detection.
3. **Self mutation On the fly Adversarial Mutator for Pipeline Hardening**: The three-strategy mutation engine is automatically operated within the CI training pipeline at each epoch where the dead-code insertion, register-level

substitution and tail-padding evasion mutators are simulated as the polymorphic malware would do to evade analysis by API-sequence analysers in CI sandbox runners.

4. **Robustness Decay Matrix for Pipeline Deployment Decisions**: Given that it reveals the accuracy of each hardening ratio against an attack vector, the mutation-rate sweep of 0% to 50% over four modeling scenarios and three attack vectors generates a structured robustness decay matrix which facilitates pipeline operators to choose a hardening ratio adapted to their threat distribution, in contrast to many current pipeline protective tools that simply report single-point accuracy figures.

5. **STRIDE-grounded threat taxonomy**: Mapping STRIDE on specific component of pipeline it produces reuseable taxonomy that already has the idea of all security categories. This helps with creating automated action against malware.

6.3 Recommendations for Future Work

6.3.1 npm Supply Chain Attack Detection

The current system is only checking newly created files after the build process and not checking the files downloaded from npm install or pip install. Over the last few years, attackers have been exploiting CI/CD pipelines for a variety of attacks before the code is even compiled: S1ngularity, Shai-Hulud, and CI/CD compromise via node-ipc are all examples. In future, before packages are being installed, the package metadata, package lifecycle scripts and unusual network requests should be detected and/or attempts to access CI secrets through package metadata.

6.3.2 Graph Neural Networks for Dependency Analysis

Existing BiLSTM model can only interpret the API calls as sequences without any dependency relations. The use of Graph Neural Networks (GCNs or GATs) in the future in order to analyze the complete dependency tree and detect risks that are not present in the transitive ones before downloading packages.

6.3.3 Continual Learning

The model is currently periodically retrained so there may be gaps when new malware emerges. Consideration should be given to using persistent updating techniques like EWC, GEM etc to learn from new threats while retaining knowledge of old ones for future use. Federated learning may also enable enterprises to share threat intelligence without being required to share sensitive data.

6.3.4 Multi-Ecosystem Support

The existing system is dedicated to Windows PE files. Containers, Linux binaries, Python packages, and Go modules also are used in modern pipelines. The framework

should be extended to enable the support of more different types of artifacts by having only a single feature extraction process for those artifacts.

6.3.5 Adversarial Attacks on the Static Branch (CNN)

The current adversarial mutation engine (Insert, Replace, Append) focuses entirely on evading the dynamic BiLSTM sequence branch. Future research should explore adversarial pixel perturbations or PE header obfuscations designed to specifically evade the CNN image branch. Analyzing how the dual-branch architecture responds when the static modality is under targeted attack will be essential for ensuring comprehensive resilience.

6.3.6 Automated Remediation and Self-Healing Pipelines

Currently, the pipeline halts and alerts the developer when an artifact is classified as malware. Future iterations could integrate Large Language Models (LLMs) or rules-based automated reasoning to generate automatic rollback pull requests or suggest secure package version bumps. This would transition the pipeline from an alerting mechanism into a fully automated "self-healing" system that requires minimal manual incident response.

Bibliography

[1] Y. Tsukamoto and A. Namatame, "Evolving neural network models," in *Proceedings of IEEE International Conference on Evolutionary Computation*, 1996, pp. 689–693. DOI: 10.1109/ICEC.1996.542685

[2] I. Harvey, P. Husb, and D. Cliff, "Incremental evolution of neural network architectures for adaptive behaviour," Aug. 1997.

[3] M. Christodorescu, S. Seshia, D. Song, and R. Bryant, "Semantics-aware malware detection," Jun. 2005, pp. 32–46, ISBN: 0-7695-2339-0. DOI: 10.1109/SP.2005.20

[4] J. Mirkovic et al., "Testing a collaborative ddos defense in a red team/blue team exercise," *IEEE Trans. Comput.*, vol. 57, no. 8, pp. 1098–1112, Aug. 2008, ISSN: 0018-9340. DOI: 10.1109/TC.2008.42 [Online]. Available: https://doi.org/10.1109/TC.2008.42

[5] R. Miikkulainen and K. O. Stanley, "Evolving neural networks," in *Proceedings of the 11th Annual Conference Companion on Genetic and Evolutionary Computation Conference: Late Breaking Papers*, ser. GECCO '09, Montreal, Québec, Canada: Association for Computing Machinery, 2009, pp. 2977–3014, ISBN: 9781605585055. DOI: 10.1145/1570256.1570410 [Online]. Available: https://doi.org/10.1145/1570256.1570410

[6] A. Applebaum, D. Miller, B. Strom, C. Korban, and R. Wolf, "Intelligent, automated red team emulation," in *Proceedings of the 32nd Annual Conference on Computer Security Applications*, ser. ACSAC '16, Los Angeles, California, USA: Association for Computing Machinery, 2016, pp. 363–373, ISBN: 9781450347716. DOI: 10.1145/2991079.2991111 [Online]. Available: https://doi.org/10.1145/2991079.2991111

[7] B. Kolosnjaji, A. Zarras, G. Webster, and C. Eckert, "Deep learning for classification of malware system call sequences," in *AI 2016: Advances in Artificial Intelligence: 29th Australasian Joint Conference, Hobart, TAS, Australia, December 5-8, 2016, Proceedings*, Hobart, TAS, Australia: Springer-Verlag, 2016, pp. 137–149, ISBN: 978-3-319-50126-0. DOI: 10.1007/978-3-319-50127-7_11 [Online]. Available: https://doi.org/10.1007/978-3-319-50127-7_11

[8] L. Chen, Y. Ye, and T. Bourlai, "Adversarial machine learning in malware detection: Arms race between evasion attack and defense," Sep. 2017, pp. 99–106. DOI: 10.1109/EISIC.2017.21

[9] A. Damodaran, F. Di Troia, C. A. Visaggio, T. Austin, and M. Stamp, "A comparison of static, dynamic, and hybrid analysis for malware detection," *Journal of Computer Virology and Hacking Techniques*, vol. 13, Feb. 2017. DOI: 10.1007/s11416-015-0261-z

[10] M. Kalash, M. Rochan, N. Mohammed, N. D. B. Bruce, Y. Wang, and F. Iqbal, "Malware classification with deep convolutional neural networks," in *2018 9th IFIP International Conference on New Technologies, Mobility and Security (NTMS)*, 2018, pp. 1–5. DOI: 10.1109/NTMS.2018.8328749 [Online]. Available: https://doi.org/10.1109/NTMS.2018.8328749

[11] L. Onwuzurike, M. Almeida, E. Mariconti, J. Blackburn, G. Stringhini, and E. De Cristofaro, "A family of droids–android malware detection via behavioral modeling: Static vs dynamic analysis," Aug. 2018. DOI: 10.1109/PST.2018.8514191

[12] A. Oliveira and R. J. Sassi, "Behavioral malware detection using deep graph convolutional neural networks," *TechRxiv Preprint*, 2019. DOI: 10.36227/techrxiv.10043090 [Online]. Available: https://doi.org/10.36227/techrxiv.10043090

[13] F. O. Catak, A. F. Yazi, O. Elezaj, and J. Ahmed, "Deep learning based sequential model for malware analysis using windows exe api calls," *PeerJ Computer Science*, vol. 6, e285, 2020. DOI: 10.7717/peerj-cs.285 [Online]. Available: https://doi.org/10.7717/peerj-cs.285

[14] S. Kotyan and D. Vargas, "Towards evolving robust neural architectures to defend from adversarial attacks," Jul. 2020, pp. 135–136. DOI: 10.1145/3377929.3389962

[15] N. Galloro, M. Polino, M. Carminati, A. Continella, and S. Zanero, "A systematical and longitudinal study of evasive behaviors in windows malware," *Comput. Secur.*, vol. 113, no. C, Feb. 2022, ISSN: 0167-4048. DOI: 10.1016/j.cose.2021.102550 [Online]. Available: https://doi.org/10.1016/j.cose.2021.102550

[16] S. A. Ganugapati and S. Prabhu, "Unifying governance, risk and controls framework using sdlc, cicd and devops," in *2023 8th International Conference on Communication and Electronics Systems (ICCES)*, 2023, pp. 1797–1802. DOI: 10.1109/ICCES57224.2023.10192730

[17] M. Marandi, A. Bertia, and S. Silas, "Implementing and automating security scanning to a devsecops ci/cd pipeline," in *2023 World Conference on Communication & Computing (WCONF)*, 2023, pp. 1–6. DOI: 10.1109/WCONF58270.2023.10235015

[18] T. Abdiukov, "Automated security testing in devsecops pipelines: Integrating ai-based vulnerability discovery and compliance validation," *World Journal of Advanced Research and Reviews*, vol. 22, pp. 2083–2093, Apr. 2024. DOI: 10.30574/wjarr.2024.22.1.1083

[19] R. Mihalache, D. Gavriluț, and D. Anton, "Real-time deep learning-based malware detection using static and dynamic features," in *Proceedings of the 16th International Conference on Agents and Artificial Intelligence - Volume 3: ICAART*, INSTICC, SciTePress, 2024, pp. 226–234, ISBN: 978-989-758-680-4. DOI: 10.5220/0012316800003636

[20] T. Shi, S. Wang, H. Li, S. Guo, and Y. Liu, "Generating adversarial samples based on the attack robust training model," Jul. 2024, pp. 896–901. DOI: 10.1109/QRS-C63300.2024.00119

[21] R. Ashtagi, C. Belani, P. Bhalerao, Y. Bhalerao, and A. Chawle, "Building resilient cicd pipelines: A devops security-first framework," in *2025 International Conference on Computational, Communication and Information Technology (ICCCIT)*, 2025, pp. 828–834. DOI: 10.1109/ICCCIT62592.2025.10927871

[22] Dikshant and Verma, *Reducing false positives with active behavioral analysis for cloud security*, Aug. 2025. DOI: 10.48550/arXiv.2508.12584

[23] G. Modalavalasa and P. Yadav, "A hybrid approach to cloud database security: Integrating dl and machine learning for threat detection and prevention," in *2025 3rd International Conference on Inventive Computing and Informatics (ICICI)*, 2025, pp. 1147–1154. DOI: 10.1109/ICICI65870.2025.11069530